```
cloudra/tools/core/command_analyzer.py:70           'ssh', 'scp', 'sftp', 'rsync',
cloudra/tools/core/command_analyzer.py:71           'wget', 'curl',
cloudra/tools/core/command_analyzer.py:72           'chmod', 'chown', 'chgrp',
cloudra/tools/core/command_analyzer.py:73           'mkfs', 'dd', 'fdisk', 'parted',
cloudra/tools/core/command_analyzer.py:74           'kill', 'pkill', 'killall',
cloudra/tools/core/command_analyzer.py:75           'shutdown', 'reboot', 'poweroff',
cloudra/tools/core/command_analyzer.py:76           'tar', 'zip', 'unzip', 'gzip', 'gunzip',
cloudra/tools/core/command_analyzer.py:77           'source', 'export',
cloudra/tools/core/command_analyzer.py:78           'pip', 'pip3', 'conda', 'npm', 'npx', 'yarn',
cloudra/tools/core/command_analyzer.py:79           'mkdir', 'touch', 'sed',
cloudra/tools/core/command_analyzer.py:80           # Windows equivalents
cloudra/tools/core/command_analyzer.py:81           'del', 'erase', 'rd', 'rmdir', 'move', 'ren', 'rename', 'format', '
taskkill', 'net',
cloudra/tools/core/command_analyzer.py:82           'icacls', 'takeown', 'cipher',
cloudra/tools/core/command_analyzer.py:83       }
cloudra/tools/core/command_analyzer.py:84
cloudra/tools/core/command_analyzer.py:85       # Shell metacharacters that indicate write/destructive behavior
cloudra/tools/core/command_analyzer.py:86       REDIRECT_PATTERNS = [
cloudra/tools/core/command_analyzer.py:87           re.compile(r'\d*[<>]'),        # >, >>, <, 2>, etc.
cloudra/tools/core/command_analyzer.py:88           re.compile(r'\d*>&'),          # >&
cloudra/tools/core/command_analyzer.py:89       ]
cloudra/tools/core/command_analyzer.py:90
cloudra/tools/core/command_analyzer.py:91       @classmethod
cloudra/tools/core/command_analyzer.py:92       def classify(cls, command: str) -> CommandAnalysis:
cloudra/tools/core/command_analyzer.py:93           """
cloudra/tools/core/command_analyzer.py:94           Parse and classify a shell command.
cloudra/tools/core/command_analyzer.py:95
cloudra/tools/core/command_analyzer.py:96           Handles pipes, redirects, variable assignments, and quoted strings.
cloudra/tools/core/command_analyzer.py:97           """
cloudra/tools/core/command_analyzer.py:98           import platform
cloudra/tools/core/command_analyzer.py:99           is_windows = platform.system() == 'Windows'
cloudra/tools/core/command_analyzer.py:100
cloudra/tools/core/command_analyzer.py:101          analysis = CommandAnalysis()
cloudra/tools/core/command_analyzer.py:102
cloudra/tools/core/command_analyzer.py:103          if not command or not command.strip():
cloudra/tools/core/command_analyzer.py:104              return analysis
cloudra/tools/core/command_analyzer.py:105
cloudra/tools/core/command_analyzer.py:106          # 1. Tokenize with shlex
cloudra/tools/core/command_analyzer.py:107          try:
cloudra/tools/core/command_analyzer.py:108              # On Windows, posix=False is usually better for cmd.exe
cloudra/tools/core/command_analyzer.py:109              tokens = shlex.split(command, comments=False, posix=not is_windows)
cloudra/tools/core/command_analyzer.py:110          except ValueError:
cloudra/tools/core/command_analyzer.py:111              # Unclosed quotes or other shlex failures: fall back to string
analysis
cloudra/tools/core/command_analyzer.py:112              return cls._fallback_classify(command)
cloudra/tools/core/command_analyzer.py:113
cloudra/tools/core/command_analyzer.py:114          # 2. Detect redirects at token level
cloudra/tools/core/command_analyzer.py:115          for tok in tokens:
cloudra/tools/core/command_analyzer.py:116              for pattern in cls.REDIRECT_PATTERNS:
cloudra/tools/core/command_analyzer.py:117                  if pattern.fullmatch(tok):
cloudra/tools/core/command_analyzer.py:118                      analysis.has_redirect = True
cloudra/tools/core/command_analyzer.py:119                      break
cloudra/tools/core/command_analyzer.py:120              # Pipes in Windows are also |
cloudra/tools/core/command_analyzer.py:121              if tok in ('|', '|&'):
```

```
122 cloudra/tools/core/command_analyzer.py:122                 analysis.has_pipe = True
123 cloudra/tools/core/command_analyzer.py:123
124 cloudra/tools/core/command_analyzer.py:124         # 3. Split by pipes to get individual command segments
125 cloudra/tools/core/command_analyzer.py:125         pipe_indices = [-1]
126 cloudra/tools/core/command_analyzer.py:126         for i, tok in enumerate(tokens):
127 cloudra/tools/core/command_analyzer.py:127             if tok == '|' or tok == '|&':
128 cloudra/tools/core/command_analyzer.py:128                 pipe_indices.append(i)
129 cloudra/tools/core/command_analyzer.py:129         pipe_indices.append(len(tokens))
130 cloudra/tools/core/command_analyzer.py:130
131 cloudra/tools/core/command_analyzer.py:131         commands = []
132 cloudra/tools/core/command_analyzer.py:132         for start, end in zip(pipe_indices[:-1], pipe_indices[1:]):
133 cloudra/tools/core/command_analyzer.py:133             segment = tokens[start + 1:end]
134 cloudra/tools/core/command_analyzer.py:134             if not segment:
135 cloudra/tools/core/command_analyzer.py:135                 continue
136 cloudra/tools/core/command_analyzer.py:136
137 cloudra/tools/core/command_analyzer.py:137             # Strip leading variable assignments (Unix style)
138 cloudra/tools/core/command_analyzer.py:138             cmd_idx = 0
139 cloudra/tools/core/command_analyzer.py:139             if not is_windows:
140 cloudra/tools/core/command_analyzer.py:140                 for idx, tok in enumerate(segment):
141 cloudra/tools/core/command_analyzer.py:141                     if '=' in tok and not tok.startswith('=') and not tok.
    endswith('='):
142 cloudra/tools/core/command_analyzer.py:142                         parts = tok.split('=', 1)
143 cloudra/tools/core/command_analyzer.py:143                         if parts[0].isidentifier():
144 cloudra/tools/core/command_analyzer.py:144                             continue
145 cloudra/tools/core/command_analyzer.py:145                     cmd_idx = idx
146 cloudra/tools/core/command_analyzer.py:146                     break
147 cloudra/tools/core/command_analyzer.py:147
148 cloudra/tools/core/command_analyzer.py:148             if cmd_idx >= len(segment):
149 cloudra/tools/core/command_analyzer.py:149                 continue
150 cloudra/tools/core/command_analyzer.py:150
151 cloudra/tools/core/command_analyzer.py:151             cmd_name = segment[cmd_idx]
152 cloudra/tools/core/command_analyzer.py:152
153 cloudra/tools/core/command_analyzer.py:153             # Handle absolute paths: /usr/bin/python or C:\Python39\python.exe
154 cloudra/tools/core/command_analyzer.py:154             # Replace backslashes for uniform processing
155 cloudra/tools/core/command_analyzer.py:155             cmd_name_clean = cmd_name.replace('\\', '/')
156 cloudra/tools/core/command_analyzer.py:156             cmd_base = cmd_name_clean.split('/')[-1]
157 cloudra/tools/core/command_analyzer.py:157
158 cloudra/tools/core/command_analyzer.py:158             # Remove .exe suffix if present
159 cloudra/tools/core/command_analyzer.py:159             if cmd_base.lower().endswith('.exe'):
160 cloudra/tools/core/command_analyzer.py:160                 cmd_base = cmd_base[:-4]
161 cloudra/tools/core/command_analyzer.py:161
162 cloudra/tools/core/command_analyzer.py:162             commands.append(cmd_base)
163 cloudra/tools/core/command_analyzer.py:163
164 cloudra/tools/core/command_analyzer.py:164         if not commands:
165 cloudra/tools/core/command_analyzer.py:165             return analysis
166 cloudra/tools/core/command_analyzer.py:166
167 cloudra/tools/core/command_analyzer.py:167         analysis.primary_cmd = commands[0]
168 cloudra/tools/core/command_analyzer.py:168         analysis.pipe_commands = commands
169 cloudra/tools/core/command_analyzer.py:169
170 cloudra/tools/core/command_analyzer.py:170         # 4. Classify each command in the pipeline
171 cloudra/tools/core/command_analyzer.py:171         perms = Permission.NONE
172 cloudra/tools/core/command_analyzer.py:172         for cmd in commands:
173 cloudra/tools/core/command_analyzer.py:173             cmd_lower = cmd.lower()
```

```
            # Remove common version suffixes for matching
            cmd_core = cmd_lower.rstrip('0123456789.')

            if cmd_lower in cls.READ_CMDS or cmd_core in cls.READ_CMDS:
                perms |= Permission.READ
            if cmd_lower in cls.WRITE_CMDS or cmd_core in cls.WRITE_CMDS:
                perms |= Permission.WRITE
            if cmd_lower in cls.SCRIPT_CMDS or cmd_core in cls.SCRIPT_CMDS:
                perms |= Permission.SCRIPT
            if cmd_lower in cls.DESTRUCTIVE_CMDS or cmd_core in cls.
DESTRUCTIVE_CMDS:
                perms |= Permission.DESTRUCTIVE

        # 5. Redirects imply write + destructive (can overwrite files)
        if analysis.has_redirect:
            perms |= Permission.WRITE | Permission.DESTRUCTIVE

        analysis.permissions = perms
        return analysis

    @classmethod
    def _fallback_classify(cls, command: str) -> CommandAnalysis:
        """
        Fallback when shlex fails (e.g. unclosed quotes).
        Uses regex-based word-boundary matching.
        """
        import platform
        is_windows = platform.system() == 'Windows'

        analysis = CommandAnalysis()

        # Detect redirects with regex
        if re.search(r'\d*[<>]', command):
            analysis.has_redirect = True

        # Extract the first "word" that looks like a command
        # Windows command regex needs to handle backslashes and drive letters
        if is_windows:
            # Matches C:\foo\bar.exe or just command
            first_word_match = re.search(r'^[\s]*([^\s|;&]+)', command)
        else:
            # Unix: handles variable assignments FOO=bar command
            first_word_match = re.search(r'^[\s]*(?:[A-Za-z_][A-Za-z0-9_]*=[^\s
]*[\s]+)*([^\s|;&]+)', command)

        if first_word_match:
            cmd_full = first_word_match.group(1)
            # Clean up Windows paths
            cmd_clean = cmd_full.replace('\\', '/')
            cmd = cmd_clean.split('/')[-1].lower()

            # Remove .exe
            if cmd.endswith('.exe'):
                cmd = cmd[:-4]
```

```
cloudra/tools/core/command_analyzer.py:226
cloudra/tools/core/command_analyzer.py:227             analysis.primary_cmd = cmd
cloudra/tools/core/command_analyzer.py:228             analysis.pipe_commands = [cmd]
cloudra/tools/core/command_analyzer.py:229
cloudra/tools/core/command_analyzer.py:230             perms = Permission.NONE
cloudra/tools/core/command_analyzer.py:231             cmd_core = cmd.rstrip('0123456789.')
cloudra/tools/core/command_analyzer.py:232
cloudra/tools/core/command_analyzer.py:233             for pool, perm in [
cloudra/tools/core/command_analyzer.py:234                 (cls.READ_CMDS, Permission.READ),
cloudra/tools/core/command_analyzer.py:235                 (cls.WRITE_CMDS, Permission.WRITE),
cloudra/tools/core/command_analyzer.py:236                 (cls.SCRIPT_CMDS, Permission.SCRIPT),
cloudra/tools/core/command_analyzer.py:237                 (cls.DESTRUCTIVE_CMDS, Permission.DESTRUCTIVE),
cloudra/tools/core/command_analyzer.py:238             ]:
cloudra/tools/core/command_analyzer.py:239                 if cmd in pool or cmd_core in pool:
cloudra/tools/core/command_analyzer.py:240                     perms |= perm
cloudra/tools/core/command_analyzer.py:241
cloudra/tools/core/command_analyzer.py:242             if analysis.has_redirect:
cloudra/tools/core/command_analyzer.py:243                 perms |= Permission.WRITE | Permission.DESTRUCTIVE
cloudra/tools/core/command_analyzer.py:244
cloudra/tools/core/command_analyzer.py:245             analysis.permissions = perms
cloudra/tools/core/command_analyzer.py:246
cloudra/tools/core/command_analyzer.py:247         return analysis
cloudra/tools/core/coord_guard.py:1 """手写坐标守卫：计算输入/几何文件中的坐标应来自工具，模型逐字键入时给出警告。
cloudra/tools/core/coord_guard.py:2
cloudra/tools/core/coord_guard.py:3 背景：机理任务中模型反复出现"手写坐标"行为——把凭记忆或凭感觉拼出的
cloudra/tools/core/coord_guard.py:4 笛卡尔坐标块直接写进 .gjf/.xyz，绕开 smiles_to_mol / conformer_search /
cloudra/tools/core/coord_guard.py:5 format_converter / gjf_writer 等结构化工具。prompt 层的禁令不够，这里在
cloudra/tools/core/coord_guard.py:6 harness 层检测并警告：text_writer / edit_file / bashtools 在写入几何或计算
cloudra/tools/core/coord_guard.py:7 输入文件前调用本模块检查内容，命中时**放行写入但在返回消息里附警告**。
cloudra/tools/core/coord_guard.py:8 刻意不硬拦——改 gjf 关键词、补 modredundant 等正当编辑不该被误伤，而且
cloudra/tools/core/coord_guard.py:9 直接拒绝只会逼模型换一条更隐蔽的路径；让它在结果里读到"这是绕过 supervisor
cloudra/tools/core/coord_guard.py:10     审查"的明确信号，才能把这个习惯纠正过来。
cloudra/tools/core/coord_guard.py:11
cloudra/tools/core/coord_guard.py:12     判定依据是"内容里出现连续坐标行"——从日志提取坐标的正当流程
cloudra/tools/core/coord_guard.py:13     （sed/grep/format_converter 管道）不会把坐标文本嵌在命令或写入内容里，
cloudra/tools/core/coord_guard.py:14     只有模型逐字键入才会。模型用 bash 手写坐标时，本质是想绕开 supervisor
cloudra/tools/core/coord_guard.py:15     对机理证据的审查、把没有计算记录背书的坐标塞进 DAG 系统，警告里会点破
cloudra/tools/core/coord_guard.py:16     这一点。
cloudra/tools/core/coord_guard.py:17     """
cloudra/tools/core/coord_guard.py:18
cloudra/tools/core/coord_guard.py:19     from __future__ import annotations
cloudra/tools/core/coord_guard.py:20
cloudra/tools/core/coord_guard.py:21     import re
cloudra/tools/core/coord_guard.py:22
cloudra/tools/core/coord_guard.py:23     # 元素符号 + 三个浮点数（允许科学计数法），一行一个原子。
cloudra/tools/core/coord_guard.py:24     _COORD_LINE = re.compile(
cloudra/tools/core/coord_guard.py:25         r"^\s*[A-Z][a-z]?\s+"
cloudra/tools/core/coord_guard.py:26         r"[-+]?\d+\.\d+(?:[eE][-+]?\d+)?\s+"
cloudra/tools/core/coord_guard.py:27         r"[-+]?\d+\.\d+(?:[eE][-+]?\d+)?\s+"
cloudra/tools/core/coord_guard.py:28         r"[-+]?\d+\.\d+(?:[eE][-+]?\d+)?\s*$"
cloudra/tools/core/coord_guard.py:29     )
cloudra/tools/core/coord_guard.py:30
```

```
cloudra/tools/core/coord_guard.py:31    # 几何/计算输入文件扩展名。刻意不含 .inp（ORCA/CP2K 输入常由 ASE 等脚本
cloudra/tools/core/coord_guard.py:32    # 程序化生成，误伤面大）与 .md 等报告文件（嵌入 xyz 块属正当内容）。
cloudra/tools/core/coord_guard.py:33    _GEOMETRY_EXTENSIONS = frozenset({".xyz", ".gjf", ".com"})
cloudra/tools/core/coord_guard.py:34
cloudra/tools/core/coord_guard.py:35    # 至少连续这么多坐标行才判定为坐标块，避免误伤零散引用。
cloudra/tools/core/coord_guard.py:36    MIN_COORD_LINES = 3
cloudra/tools/core/coord_guard.py:37
cloudra/tools/core/coord_guard.py:38    # 结构化坐标来源提示 + 绕开审查的定性：手写坐标不与计算记录绑定，
cloudra/tools/core/coord_guard.py:39    # 挂 DAG 证据时可能不被采信，也绕过了 supervisor 对证据等级的审查。
cloudra/tools/core/coord_guard.py:40    _GUIDANCE = (
cloudra/tools/core/coord_guard.py:41        "Coordinates should come from tools; do not type them by hand or transcribe
from memory: "
cloudra/tools/core/coord_guard.py:42        "use smiles_to_mol / conformer_search for structure generation; extract
coordinates from compute logs with "
cloudra/tools/core/coord_guard.py:43        "format_converter (log→xyz) or gaussian_log_parser; prefer gjf_writer for
Gaussian input; "
cloudra/tools/core/coord_guard.py:44        "geometry tweaks should use the internal_coordinates Z-matrix flow. "
cloudra/tools/core/coord_guard.py:45        "Hand-writing coordinates with bash means trying to bypass Supervisor review of
 mechanism evidence: handwritten coordinates are not bound "
cloudra/tools/core/coord_guard.py:46        "to any compute record and may not be accepted as DAG evidence. "
cloudra/tools/core/coord_guard.py:47        "Do not use bash to bypass chemistry tools, and do not try to bypass the DAG
system."
cloudra/tools/core/coord_guard.py:48    )
cloudra/tools/core/coord_guard.py:49
cloudra/tools/core/coord_guard.py:50
cloudra/tools/core/coord_guard.py:51    def count_consecutive_coord_lines(text: str) -> int:
cloudra/tools/core/coord_guard.py:52        """文本中最长的连续坐标行数。"""
cloudra/tools/core/coord_guard.py:53        best = run = 0
cloudra/tools/core/coord_guard.py:54        for line in text.splitlines():
cloudra/tools/core/coord_guard.py:55            if _COORD_LINE.match(line):
cloudra/tools/core/coord_guard.py:56                run += 1
cloudra/tools/core/coord_guard.py:57                if run > best:
cloudra/tools/core/coord_guard.py:58                    best = run
cloudra/tools/core/coord_guard.py:59            else:
cloudra/tools/core/coord_guard.py:60                run = 0
cloudra/tools/core/coord_guard.py:61        return best
cloudra/tools/core/coord_guard.py:62
cloudra/tools/core/coord_guard.py:63
cloudra/tools/core/coord_guard.py:64    def is_geometry_path(path: str) -> bool:
cloudra/tools/core/coord_guard.py:65        lowered = path.strip().lower()
cloudra/tools/core/coord_guard.py:66        return any(lowered.endswith(ext) for ext in _GEOMETRY_EXTENSIONS)
cloudra/tools/core/coord_guard.py:67
cloudra/tools/core/coord_guard.py:68
cloudra/tools/core/coord_guard.py:69    def handwritten_coord_warning(text: str, *, path: str | None) -> str | None:
cloudra/tools/core/coord_guard.py:70        """返回应附加到工具结果里的警告消息；无嫌疑返回 None。
cloudra/tools/core/coord_guard.py:71
cloudra/tools/core/coord_guard.py:72        调用方**照常执行写入**，只把警告附在返回消息里让模型意识到行为不当。
cloudra/tools/core/coord_guard.py:73        path 给定文件路径时只检查几何/计算输入文件；path 为 None（bash 命令）
cloudra/tools/core/coord_guard.py:74        时要求命令同时提及几何文件扩展名才警告，避免误伤展示性输出。
cloudra/tools/core/coord_guard.py:75        """
cloudra/tools/core/coord_guard.py:76        if count_consecutive_coord_lines(text) < MIN_COORD_LINES:
cloudra/tools/core/coord_guard.py:77            return None
cloudra/tools/core/coord_guard.py:78        if path is not None:
cloudra/tools/core/coord_guard.py:79            if not is_geometry_path(path):
cloudra/tools/core/coord_guard.py:80                return None
cloudra/tools/core/coord_guard.py:81            target = path
cloudra/tools/core/coord_guard.py:82        else:
```

```
cloudra/tools/core/coord_guard.py:83            if not any(ext in text.lower() for ext in _GEOMETRY_EXTENSIONS):
cloudra/tools/core/coord_guard.py:84                return None
cloudra/tools/core/coord_guard.py:85            target = "a geometry/compute input file"
cloudra/tools/core/coord_guard.py:86        return f"[Warning: suspected handwritten coordinates] Wrote a typed-in coordinate block to {target}. {_GUIDANCE}"
cloudra/tools/core/delegation_guard.py:1    """Coordinator 自行下场计算的轻量提示（软警告，不硬拦）。
cloudra/tools/core/delegation_guard.py:2
cloudra/tools/core/delegation_guard.py:3    背景：Coordinator 故意持有工具全集，方便简单任务自己做；但长上下文下
cloudra/tools/core/delegation_guard.py:4    容易把「自己做」放大成默认路径，本该派给 dft / small_mol / md 的正式计算
cloudra/tools/core/delegation_guard.py:5    也自己 submit_compute_job。这里只在成功提交时附一句提醒，放行执行。
cloudra/tools/core/delegation_guard.py:6    """
cloudra/tools/core/delegation_guard.py:7
cloudra/tools/core/delegation_guard.py:8    from __future__ import annotations
cloudra/tools/core/delegation_guard.py:9
cloudra/tools/core/delegation_guard.py:10   from typing import Any
cloudra/tools/core/delegation_guard.py:11
cloudra/tools/core/delegation_guard.py:12   _GUIDANCE = (
cloudra/tools/core/delegation_guard.py:13       "You are the Coordinator and are calling submit_compute_job yourself. "
cloudra/tools/core/delegation_guard.py:14       "Quick checks (seconds) can be done yourself; for formal or long quantum jobs, prefer dispatching "
cloudra/tools/core/delegation_guard.py:15       "dft / small_mol / md sub-agents, and include file paths, goals, and acceptance criteria in the dispatch. "
cloudra/tools/core/delegation_guard.py:16       "This submission was executed as usual."
cloudra/tools/core/delegation_guard.py:17   )
cloudra/tools/core/delegation_guard.py:18
cloudra/tools/core/delegation_guard.py:19
cloudra/tools/core/delegation_guard.py:20   def coordinator_self_compute_warning(agent: Any) -> str | None:
cloudra/tools/core/delegation_guard.py:21       """Coordinator 自行提交计算时返回提示文案；其它角色返回 None。"""
cloudra/tools/core/delegation_guard.py:22       if getattr(agent, "type", None) != "coordinator":
cloudra/tools/core/delegation_guard.py:23           return None
cloudra/tools/core/delegation_guard.py:24       return f"[Note: Coordinator submitted compute themselves] {_GUIDANCE}"
cloudra/tools/core/gaussian_frames.py:1 """Gaussian 日志关键帧提取：任务类型识别 + 逐帧结构/能量/频率导出。
cloudra/tools/core/gaussian_frames.py:2
cloudra/tools/core/gaussian_frames.py:3 让 agent 不必手写正则啃 Gaussian 日志：
cloudra/tools/core/gaussian_frames.py:4
cloudra/tools/core/gaussian_frames.py:5 - opt / freq / scan 走 cclib（scfenergies / atomcoords / vibfreqs / optdone / scanenergies / scanparm / scancoords），支持良好；
cloudra/tools/core/gaussian_frames.py:7 - IRC 为自写解析器：cclib 1.8.x 只对 IRC 做收敛记账（optstatus），不提取反应坐标与路径点，且其 scfenergies/atomcoords 会混入 predictor-corrector 微迭代的中间点，因此 IRC 分支完全不使用 cclib 的逐点序列。
cloudra/tools/core/gaussian_frames.py:10
cloudra/tools/core/gaussian_frames.py:11    本模块为纯函数，不依赖 cloudra 的 Tool 体系；cclib / numpy 在函数内惰性导入。
cloudra/tools/core/gaussian_frames.py:12    """
cloudra/tools/core/gaussian_frames.py:13
cloudra/tools/core/gaussian_frames.py:14    from __future__ import annotations
cloudra/tools/core/gaussian_frames.py:15
cloudra/tools/core/gaussian_frames.py:16    import csv
cloudra/tools/core/gaussian_frames.py:17    import os
cloudra/tools/core/gaussian_frames.py:18    import re
cloudra/tools/core/gaussian_frames.py:19    from dataclasses import dataclass, field
cloudra/tools/core/gaussian_frames.py:20
cloudra/tools/core/gaussian_frames.py:21    EV_PER_HARTREE = 27.211386245988
cloudra/tools/core/gaussian_frames.py:22
cloudra/tools/core/gaussian_frames.py:23
cloudra/tools/core/gaussian_frames.py:24    @dataclass
```

```
cloudra/tools/core/gaussian_frames.py:25    class FrameRecord:
cloudra/tools/core/gaussian_frames.py:26        """一个关键帧：优化步 / 扫描点 / IRC 路径点 / 单点结构。"""
cloudra/tools/core/gaussian_frames.py:27
cloudra/tools/core/gaussian_frames.py:28        frame: int
cloudra/tools/core/gaussian_frames.py:29        path_number: int | None = None          # IRC 路径号（1=正向 2=反向），其余任务为 None
cloudra/tools/core/gaussian_frames.py:30        reaction_coordinate: float | None = None  # IRC 反应坐标 / 扫描参数值
cloudra/tools/core/gaussian_frames.py:31        scf_energy_hartree: float | None = None
cloudra/tools/core/gaussian_frames.py:32        frequencies_cm: list[float] = field(default_factory=list)  # 有符号，负值=虚频
cloudra/tools/core/gaussian_frames.py:33        xyz: str = ""                             # 标准 XYZ 块（原子数行+注释行+坐标行）
cloudra/tools/core/gaussian_frames.py:34        converged: bool | None = None            # 末帧/端点帧的收敛标记，其余为 None
cloudra/tools/core/gaussian_frames.py:35
cloudra/tools/core/gaussian_frames.py:36
cloudra/tools/core/gaussian_frames.py:37    # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:38    # 任务类型识别
cloudra/tools/core/gaussian_frames.py:39    # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:40
cloudra/tools/core/gaussian_frames.py:41    def _read_text(log_path: str) -> str:
cloudra/tools/core/gaussian_frames.py:42        with open(log_path, encoding="utf-8", errors="replace") as f:
cloudra/tools/core/gaussian_frames.py:43            return f.read()
cloudra/tools/core/gaussian_frames.py:44
cloudra/tools/core/gaussian_frames.py:45
cloudra/tools/core/gaussian_frames.py:46    def _collect_route_card(text: str) -> str:
cloudra/tools/core/gaussian_frames.py:47        """收集日志中回显的路由行（` #` 开头，可能跨行缠绕），拼接为小写字符串。
cloudra/tools/core/gaussian_frames.py:48
cloudra/tools/core/gaussian_frames.py:49        Gaussian 的路由回显块中每行以 " #" 开头；跨行缠绕多为行写满后从 token
cloudra/tools/core/gaussian_frames.py:50        中间断开（如 "stepsize=10,m" / "axpoint=100"），因此同一块内直接无空格
cloudra/tools/core/gaussian_frames.py:51        拼接。IRC 日志里的 " # OF POINTS ALONG THE PATH" 等计数行不是路由，剔除。
cloudra/tools/core/gaussian_frames.py:52        """
cloudra/tools/core/gaussian_frames.py:53        blocks: list[str] = []
cloudra/tools/core/gaussian_frames.py:54        current: list[str] = []
cloudra/tools/core/gaussian_frames.py:55        for line in text.splitlines():
cloudra/tools/core/gaussian_frames.py:56            stripped = line.strip()
cloudra/tools/core/gaussian_frames.py:57            if stripped.startswith("#") and not stripped.upper().startswith("# OF"):
cloudra/tools/core/gaussian_frames.py:58                current.append(stripped[1:].strip())
cloudra/tools/core/gaussian_frames.py:59            else:
cloudra/tools/core/gaussian_frames.py:60                if current:
cloudra/tools/core/gaussian_frames.py:61                    blocks.append("".join(current))
cloudra/tools/core/gaussian_frames.py:62                    current = []
cloudra/tools/core/gaussian_frames.py:63        if current:
cloudra/tools/core/gaussian_frames.py:64            blocks.append("".join(current))
cloudra/tools/core/gaussian_frames.py:65        return " ".join(blocks).lower()
cloudra/tools/core/gaussian_frames.py:66
cloudra/tools/core/gaussian_frames.py:67
cloudra/tools/core/gaussian_frames.py:68    def detect_job_types(log_path: str) -> set[str]:
cloudra/tools/core/gaussian_frames.py:69        """识别 Gaussian 日志的任务类型，返回 {"opt","ts","freq","scan","irc","sp"} 的子集。
cloudra/tools/core/gaussian_frames.py:70
cloudra/tools/core/gaussian_frames.py:71        依据路由卡（` #` 回显行）+ 结构标记兜底（IRC 的 "IRC-IRC-IRC" 块与
cloudra/tools/core/gaussian_frames.py:72        "Summary of reaction path following"，扫描的 "Summary of Optimized
cloudra/tools/core/gaussian_frames.py:73        Potential Surface Scan" 等）。`ts` 只在路由中出现 opt=(ts...) / opt=ts 时给出。
cloudra/tools/core/gaussian_frames.py:74
cloudra/tools/core/gaussian_frames.py:75        限制：Link1 多步任务不拆分 section，按整文件取合集（v1）。
cloudra/tools/core/gaussian_frames.py:76        """
```

```
77 cloudra/tools/core/gaussian_frames.py:77        text = _read_text(log_path)
78 cloudra/tools/core/gaussian_frames.py:78        route = _collect_route_card(text)
79 cloudra/tools/core/gaussian_frames.py:79        jobs: set[str] = set()
80 cloudra/tools/core/gaussian_frames.py:80
81 cloudra/tools/core/gaussian_frames.py:81        if (
82 cloudra/tools/core/gaussian_frames.py:82            re.search(r"\birc\b|irc\s*[=(]", route)
83 cloudra/tools/core/gaussian_frames.py:83            or "IRC-IRC-IRC" in text
84 cloudra/tools/core/gaussian_frames.py:84            or "Summary of reaction path following" in text
85 cloudra/tools/core/gaussian_frames.py:85        ):
86 cloudra/tools/core/gaussian_frames.py:86            jobs.add("irc")
87 cloudra/tools/core/gaussian_frames.py:87        if re.search(r"\bscan\b", route) or "Summary of Optimized Potential Surface
    Scan" in text:
88 cloudra/tools/core/gaussian_frames.py:88            jobs.add("scan")
89 cloudra/tools/core/gaussian_frames.py:89        if re.search(r"\bopt\b|opt\s*=", route) or "Optimization completed" in text
   :
90 cloudra/tools/core/gaussian_frames.py:90            jobs.add("opt")
91 cloudra/tools/core/gaussian_frames.py:91        if re.search(r"opt\s*=\s*\(?\s*ts\b", route) or re.search(r"opt\s*=\s*\([^)
   ]*\bts\b", route):
92 cloudra/tools/core/gaussian_frames.py:92            jobs.add("ts")
93 cloudra/tools/core/gaussian_frames.py:93        if re.search(r"\bfreq", route) or "Harmonic frequencies" in text:
94 cloudra/tools/core/gaussian_frames.py:94            jobs.add("freq")
95 cloudra/tools/core/gaussian_frames.py:95        if not jobs & {"irc", "scan", "opt", "ts"}:
96 cloudra/tools/core/gaussian_frames.py:96            # 无几何移动的任务（含 freq-only）视为单点
97 cloudra/tools/core/gaussian_frames.py:97            jobs.add("sp")
98 cloudra/tools/core/gaussian_frames.py:98        return jobs
99 cloudra/tools/core/gaussian_frames.py:99
100 cloudra/tools/core/gaussian_frames.py:100
101 cloudra/tools/core/gaussian_frames.py:101   # ---------------------------------------------------------------------------
102 cloudra/tools/core/gaussian_frames.py:102   # XYZ 生成
103 cloudra/tools/core/gaussian_frames.py:103   # ---------------------------------------------------------------------------
104 cloudra/tools/core/gaussian_frames.py:104
105 cloudra/tools/core/gaussian_frames.py:105   def _coords_to_xyz(atomnos, coords, comment: str = "") -> str:
106 cloudra/tools/core/gaussian_frames.py:106       """atomnos + 坐标（Angstrom）生成标准 XYZ 块。"""
107 cloudra/tools/core/gaussian_frames.py:107       from cclib.parser.utils import PeriodicTable
108 cloudra/tools/core/gaussian_frames.py:108
109 cloudra/tools/core/gaussian_frames.py:109       pt = PeriodicTable()
110 cloudra/tools/core/gaussian_frames.py:110       lines = [str(len(atomnos)), comment]
111 cloudra/tools/core/gaussian_frames.py:111       for z, (x, y, zc) in zip(atomnos, coords, strict=True):
112 cloudra/tools/core/gaussian_frames.py:112           lines.append(f"{pt.element[int(z)]:<2} {x:>15.8f} {y:>15.8f} {zc:>15.8f
    }")
113 cloudra/tools/core/gaussian_frames.py:113       return "\n".join(lines)
114 cloudra/tools/core/gaussian_frames.py:114
115 cloudra/tools/core/gaussian_frames.py:115
116 cloudra/tools/core/gaussian_frames.py:116   # ---------------------------------------------------------------------------
117 cloudra/tools/core/gaussian_frames.py:117   # IRC 解析（自写，不走 cclib 逐点序列）
118 cloudra/tools/core/gaussian_frames.py:118   # ---------------------------------------------------------------------------
119 cloudra/tools/core/gaussian_frames.py:119
120 cloudra/tools/core/gaussian_frames.py:120   _POINT_HEADER_RE = re.compile(r"Point Number:\s*(\d+)\s+Path Number:\s*(\d+)")
121 cloudra/tools/core/gaussian_frames.py:121   _NET_RX_RE = re.compile(r"NET REACTION COORDINATE UP TO THIS POINT =\s*(-?[\d
    .]+)")
122 cloudra/tools/core/gaussian_frames.py:122   _TS_ENERGY_RE = re.compile(r"Energies reported relative to the TS energy of\s
    *(-?[\d.]+)")
123 cloudra/tools/core/gaussian_frames.py:123   _SUMMARY_TABLE_RE = re.compile(
124 cloudra/tools/core/gaussian_frames.py:124       r"Summary of reaction path following\s*\n\s*-+\s*\n\s*Energy\s+RxCoord\s*\n
    (.*?)\n\s*-+",
125 cloudra/tools/core/gaussian_frames.py:125       re.S,
126 cloudra/tools/core/gaussian_frames.py:126   )
127 cloudra/tools/core/gaussian_frames.py:127
128 cloudra/tools/core/gaussian_frames.py:128
```

```
cloudra/tools/core/gaussian_frames.py:129   def _is_dash_line(line: str) -> bool:
cloudra/tools/core/gaussian_frames.py:130       s = line.strip()
cloudra/tools/core/gaussian_frames.py:131       return len(s) > 3 and set(s) == {"-"}
cloudra/tools/core/gaussian_frames.py:132
cloudra/tools/core/gaussian_frames.py:133
cloudra/tools/core/gaussian_frames.py:134   def _parse_coord_block_after(lines: list[str], start: int, has_type_col: bool):
cloudra/tools/core/gaussian_frames.py:135       """从 lines[start] 之后第一个双虚线表的坐标区解析 (atomno, x, y, z) 列表。"""
cloudra/tools/core/gaussian_frames.py:136       dash_seen = 0
cloudra/tools/core/gaussian_frames.py:137       rows = []
cloudra/tools/core/gaussian_frames.py:138       for i in range(start, len(lines)):
cloudra/tools/core/gaussian_frames.py:139           line = lines[i]
cloudra/tools/core/gaussian_frames.py:140           if _is_dash_line(line):
cloudra/tools/core/gaussian_frames.py:141               dash_seen += 1
cloudra/tools/core/gaussian_frames.py:142               if dash_seen == 3:
cloudra/tools/core/gaussian_frames.py:143                   break
cloudra/tools/core/gaussian_frames.py:144               continue
cloudra/tools/core/gaussian_frames.py:145           if dash_seen == 2:
cloudra/tools/core/gaussian_frames.py:146               parts = line.split()
cloudra/tools/core/gaussian_frames.py:147               if has_type_col and len(parts) >= 6:
cloudra/tools/core/gaussian_frames.py:148                   rows.append((int(parts[1]), float(parts[3]), float(parts[4]),
    float(parts[5])))
cloudra/tools/core/gaussian_frames.py:149               elif not has_type_col and len(parts) == 5:
cloudra/tools/core/gaussian_frames.py:150                   rows.append((int(parts[1]), float(parts[2]), float(parts[3]),
    float(parts[4])))
cloudra/tools/core/gaussian_frames.py:151       return rows or None
cloudra/tools/core/gaussian_frames.py:152
cloudra/tools/core/gaussian_frames.py:153
cloudra/tools/core/gaussian_frames.py:154   def _parse_current_structure(segment: str):
cloudra/tools/core/gaussian_frames.py:155       """解析点段内 "CURRENT STRUCTURE" 的 Cartesian 坐标块（无 Type 列，5 列）。"""
cloudra/tools/core/gaussian_frames.py:156       lines = segment.splitlines()
cloudra/tools/core/gaussian_frames.py:157       for i, line in enumerate(lines):
cloudra/tools/core/gaussian_frames.py:158           if "CURRENT STRUCTURE" in line:
cloudra/tools/core/gaussian_frames.py:159               return _parse_coord_block_after(lines, i + 1, has_type_col=False)
cloudra/tools/core/gaussian_frames.py:160       return None
cloudra/tools/core/gaussian_frames.py:161
cloudra/tools/core/gaussian_frames.py:162
cloudra/tools/core/gaussian_frames.py:163   def _parse_last_input_orientation(region: str):
cloudra/tools/core/gaussian_frames.py:164       """解析文本区域内最后一个 "Input orientation:" 坐标块（含 Type 列，6 列）。"""
cloudra/tools/core/gaussian_frames.py:165       lines = region.splitlines()
cloudra/tools/core/gaussian_frames.py:166       last_idx = None
cloudra/tools/core/gaussian_frames.py:167       for i, line in enumerate(lines):
cloudra/tools/core/gaussian_frames.py:168           if "Input orientation:" in line:
cloudra/tools/core/gaussian_frames.py:169               last_idx = i
cloudra/tools/core/gaussian_frames.py:170       if last_idx is None:
cloudra/tools/core/gaussian_frames.py:171           return None
cloudra/tools/core/gaussian_frames.py:172       return _parse_coord_block_after(lines, last_idx + 1, has_type_col=True)
cloudra/tools/core/gaussian_frames.py:173
cloudra/tools/core/gaussian_frames.py:174
cloudra/tools/core/gaussian_frames.py:175   def _parse_irc_summary_table(text: str):
cloudra/tools/core/gaussian_frames.py:176       """解析结尾汇总表，返回 (ts_energy_hartree, [(energy_rel, rx_coord), ...])。
cloudra/tools/core/gaussian_frames.py:177
cloudra/tools/core/gaussian_frames.py:178       表中能量为相对 TS 能量的差值，行按反应坐标从负到正排列（TS 行为 rx=0）。
cloudra/tools/core/gaussian_frames.py:179       """
cloudra/tools/core/gaussian_frames.py:180       m_ts = _TS_ENERGY_RE.search(text)
```

```
cloudra/tools/core/gaussian_frames.py:181       m_tab = _SUMMARY_TABLE_RE.search(text)
cloudra/tools/core/gaussian_frames.py:182       if not m_ts or not m_tab:
cloudra/tools/core/gaussian_frames.py:183           raise ValueError("IRC 日志缺少 'Summary of reaction path following' 汇总表，无法提取反应坐标与能量。")
cloudra/tools/core/gaussian_frames.py:184       rows = []
cloudra/tools/core/gaussian_frames.py:185       for line in m_tab.group(1).splitlines():
cloudra/tools/core/gaussian_frames.py:186           parts = line.split()
cloudra/tools/core/gaussian_frames.py:187           if len(parts) == 3:
cloudra/tools/core/gaussian_frames.py:188               rows.append((float(parts[1]), float(parts[2])))
cloudra/tools/core/gaussian_frames.py:189       if not rows:
cloudra/tools/core/gaussian_frames.py:190           raise ValueError("IRC 汇总表为空。")
cloudra/tools/core/gaussian_frames.py:191       return float(m_ts.group(1)), rows
cloudra/tools/core/gaussian_frames.py:192
cloudra/tools/core/gaussian_frames.py:193
cloudra/tools/core/gaussian_frames.py:194   def _irc_direction_converged(text: str, direction: str) -> bool:
cloudra/tools/core/gaussian_frames.py:195       """判断 IRC 某个方向（FORWARD/REVERSE）是否收敛到极小点。
cloudra/tools/core/gaussian_frames.py:196
cloudra/tools/core/gaussian_frames.py:197       Gaussian 在路径自然终止于 PES 极小点时直接打印
cloudra/tools/core/gaussian_frames.py:198       "Calculation of XXX path complete."；若因步数耗尽终止则先打印
cloudra/tools/core/gaussian_frames.py:199       "Maximum number of steps reached."。cclib 的 optstatus 对 IRC 点一律记为
cloudra/tools/core/gaussian_frames.py:200       DONE，无法区分这两种情形，故以文本标记为准。
cloudra/tools/core/gaussian_frames.py:201       """
cloudra/tools/core/gaussian_frames.py:202       idx = text.find(f"Calculation of {direction} path complete.")
cloudra/tools/core/gaussian_frames.py:203       if idx < 0:
cloudra/tools/core/gaussian_frames.py:204           return False
cloudra/tools/core/gaussian_frames.py:205       window = text[max(0, idx - 500):idx]
cloudra/tools/core/gaussian_frames.py:206       return "Maximum number of steps reached" not in window
cloudra/tools/core/gaussian_frames.py:207
cloudra/tools/core/gaussian_frames.py:208
cloudra/tools/core/gaussian_frames.py:209   def _parse_irc(text: str):
cloudra/tools/core/gaussian_frames.py:210       """解析 IRC 日志，返回 (frames, meta)。
cloudra/tools/core/gaussian_frames.py:211
cloudra/tools/core/gaussian_frames.py:212       - 反应坐标与能量：结尾 "Summary of reaction path following" 汇总表
cloudra/tools/core/gaussian_frames.py:213         （能量 = TS 能量 + 相对能量）；
cloudra/tools/core/gaussian_frames.py:214       - 坐标：每个 "Point Number: N / Path Number: M" 标记后的 CURRENT STRUCTURE
cloudra/tools/core/gaussian_frames.py:215         坐标块（即 corrector 收敛后的该点结构；点段内的 Input orientation 是
cloudra/tools/core/gaussian_frames.py:216         下一点的 predictor 猜测结构，不采用）。Point 0（TS）无 CURRENT STRUCTURE
cloudra/tools/core/gaussian_frames.py:217         块时，回退到该标记之前最后一个 Input orientation 块，仍无则 xyz 留空；
cloudra/tools/core/gaussian_frames.py:218       - 汇总表按 rx 从负到正排列，通过各点 "NET REACTION COORDINATE"（到 TS 的
cloudra/tools/core/gaussian_frames.py:219         距离，恒正）与 |rx| 匹配，确定正/负侧对应哪个 Path Number；
cloudra/tools/core/gaussian_frames.py:220       - 输出帧按反应坐标升序排列（反向端点 → TS → 正向端点）。
cloudra/tools/core/gaussian_frames.py:221       """
cloudra/tools/core/gaussian_frames.py:222       ts_energy, rows = _parse_irc_summary_table(text)
cloudra/tools/core/gaussian_frames.py:223
cloudra/tools/core/gaussian_frames.py:224       headers = list(_POINT_HEADER_RE.finditer(text))
cloudra/tools/core/gaussian_frames.py:225       if not headers:
cloudra/tools/core/gaussian_frames.py:226           raise ValueError("IRC 日志中未找到 'Point Number: N / Path Number: M' 标记。")
cloudra/tools/core/gaussian_frames.py:227
cloudra/tools/core/gaussian_frames.py:228       # 每个点段：从当前标记到下一标记
cloudra/tools/core/gaussian_frames.py:229       points: dict[int, list[dict]] = {1: [], 2: []}
cloudra/tools/core/gaussian_frames.py:230       for i, m in enumerate(headers):
cloudra/tools/core/gaussian_frames.py:231           point_no, path_no = int(m.group(1)), int(m.group(2))
cloudra/tools/core/gaussian_frames.py:232           seg_end = headers[i + 1].start() if i + 1 < len(headers) else len(text)
```

```
cloudra/tools/core/gaussian_frames.py:233           segment = text[m.end():seg_end]
cloudra/tools/core/gaussian_frames.py:234           coords = _parse_current_structure(segment)
cloudra/tools/core/gaussian_frames.py:235           if point_no == 0 and coords is None:
cloudra/tools/core/gaussian_frames.py:236               # TS 起始点：尝试取标记之前（初始结构 SCF）的最后一个 Input orientation
cloudra/tools/core/gaussian_frames.py:237               coords = _parse_last_input_orientation(text[:m.start()])
cloudra/tools/core/gaussian_frames.py:238           m_net = _NET_RX_RE.search(segment)
cloudra/tools/core/gaussian_frames.py:239           points.setdefault(path_no, []).append({
cloudra/tools/core/gaussian_frames.py:240               "point": point_no,
cloudra/tools/core/gaussian_frames.py:241               "path": path_no,
cloudra/tools/core/gaussian_frames.py:242               "net_rx": float(m_net.group(1)) if m_net else (0.0 if point_no == 0
     else None),
cloudra/tools/core/gaussian_frames.py:243               "coords": coords,
cloudra/tools/core/gaussian_frames.py:244           })
cloudra/tools/core/gaussian_frames.py:245
cloudra/tools/core/gaussian_frames.py:246       # 拆分汇总表：rx=0 为 TS 行，负/正两侧按到 TS 距离升序
cloudra/tools/core/gaussian_frames.py:247       neg_rows = sorted((r for r in rows if r[1] < 0), key=lambda r: -r[1])
cloudra/tools/core/gaussian_frames.py:248       pos_rows = sorted((r for r in rows if r[1] > 0), key=lambda r: r[1])
cloudra/tools/core/gaussian_frames.py:249       ts_rows = [r for r in rows if r[1] == 0]
cloudra/tools/core/gaussian_frames.py:250
cloudra/tools/core/gaussian_frames.py:251       def _mismatch(pts, side_rows):
cloudra/tools/core/gaussian_frames.py:252           n = min(len(pts), len(side_rows))
cloudra/tools/core/gaussian_frames.py:253           return sum(
cloudra/tools/core/gaussian_frames.py:254               abs((pts[i]["net_rx"] or 0.0) - abs(side_rows[i][1]))
cloudra/tools/core/gaussian_frames.py:255               for i in range(n)
cloudra/tools/core/gaussian_frames.py:256           )
cloudra/tools/core/gaussian_frames.py:257
cloudra/tools/core/gaussian_frames.py:258       # Path 1 的点列与负/正侧各匹配一次，取总误差小的归属
cloudra/tools/core/gaussian_frames.py:259       pts1 = sorted(points.get(1, []), key=lambda p: p["point"])
cloudra/tools/core/gaussian_frames.py:260       pts2 = sorted(points.get(2, []), key=lambda p: p["point"])
cloudra/tools/core/gaussian_frames.py:261       pts1 = [p for p in pts1 if p["point"] > 0]
cloudra/tools/core/gaussian_frames.py:262       pts2 = [p for p in pts2 if p["point"] > 0]
cloudra/tools/core/gaussian_frames.py:263       score_a = _mismatch(pts1, pos_rows) + _mismatch(pts2, neg_rows)
cloudra/tools/core/gaussian_frames.py:264       score_b = _mismatch(pts1, neg_rows) + _mismatch(pts2, pos_rows)
cloudra/tools/core/gaussian_frames.py:265       path_rows = {1: pos_rows, 2: neg_rows} if score_a <= score_b else {1:
    neg_rows, 2: pos_rows}
cloudra/tools/core/gaussian_frames.py:266
cloudra/tools/core/gaussian_frames.py:267       conv = {
cloudra/tools/core/gaussian_frames.py:268           1: _irc_direction_converged(text, "FORWARD"),
cloudra/tools/core/gaussian_frames.py:269           2: _irc_direction_converged(text, "REVERSE"),
cloudra/tools/core/gaussian_frames.py:270       }
cloudra/tools/core/gaussian_frames.py:271
cloudra/tools/core/gaussian_frames.py:272       def _make_frame(frame_idx, path_no, pt, row):
cloudra/tools/core/gaussian_frames.py:273           energy_rel, rx = row
cloudra/tools/core/gaussian_frames.py:274           energy = ts_energy + energy_rel
cloudra/tools/core/gaussian_frames.py:275           coords = pt["coords"] if pt else None
cloudra/tools/core/gaussian_frames.py:276           atomnos = [c[0] for c in coords] if coords else None
cloudra/tools/core/gaussian_frames.py:277           xyz = ""
cloudra/tools/core/gaussian_frames.py:278           if coords:
cloudra/tools/core/gaussian_frames.py:279               xyz = _coords_to_xyz(
cloudra/tools/core/gaussian_frames.py:280                   atomnos,
cloudra/tools/core/gaussian_frames.py:281                   [c[1:] for c in coords],
cloudra/tools/core/gaussian_frames.py:282                   comment=f"irc point={pt['point']} path={path_no} rx={rx:.5f} E
    ={energy:.8f} Eh",
cloudra/tools/core/gaussian_frames.py:283               )
cloudra/tools/core/gaussian_frames.py:284           return FrameRecord(
```

```
cloudra/tools/core/gaussian_frames.py:285               frame=frame_idx,
cloudra/tools/core/gaussian_frames.py:286               path_number=path_no,
cloudra/tools/core/gaussian_frames.py:287               reaction_coordinate=rx,
cloudra/tools/core/gaussian_frames.py:288               scf_energy_hartree=energy,
cloudra/tools/core/gaussian_frames.py:289               xyz=xyz,
cloudra/tools/core/gaussian_frames.py:290               converged=None,
cloudra/tools/core/gaussian_frames.py:291           )
cloudra/tools/core/gaussian_frames.py:292
cloudra/tools/core/gaussian_frames.py:293       frames: list[FrameRecord] = []
cloudra/tools/core/gaussian_frames.py:294       idx = 0
cloudra/tools/core/gaussian_frames.py:295       # 反向侧（rx 更负）：点号从大到小
cloudra/tools/core/gaussian_frames.py:296       for path_no in (1, 2):
cloudra/tools/core/gaussian_frames.py:297           side = path_rows[path_no]
cloudra/tools/core/gaussian_frames.py:298           if side and side[0][1] < 0:
cloudra/tools/core/gaussian_frames.py:299               pts = pts1 if path_no == 1 else pts2
cloudra/tools/core/gaussian_frames.py:300               for i in range(len(side) - 1, -1, -1):
cloudra/tools/core/gaussian_frames.py:301                   pt = pts[i] if i < len(pts) else None
cloudra/tools/core/gaussian_frames.py:302                   fr = _make_frame(idx, path_no, pt or {"point": i + 1, "coords":
     None}, side[i])
cloudra/tools/core/gaussian_frames.py:303                   if i == len(side) - 1:
cloudra/tools/core/gaussian_frames.py:304                       fr.converged = conv[path_no]  # 该方向端点
cloudra/tools/core/gaussian_frames.py:305                   frames.append(fr)
cloudra/tools/core/gaussian_frames.py:306                   idx += 1
cloudra/tools/core/gaussian_frames.py:307       # TS
cloudra/tools/core/gaussian_frames.py:308       ts_pt = next((p for p in points.get(1, []) if p["point"] == 0), None)
cloudra/tools/core/gaussian_frames.py:309       ts_row = ts_rows[0] if ts_rows else (0.0, 0.0)
cloudra/tools/core/gaussian_frames.py:310       frames.append(_make_frame(idx, ts_pt["path"] if ts_pt else 1,
cloudra/tools/core/gaussian_frames.py:311                                 ts_pt or {"point": 0, "coords": None}, ts_row))
cloudra/tools/core/gaussian_frames.py:312       idx += 1
cloudra/tools/core/gaussian_frames.py:313       # 正向侧（rx 更正）：点号从小到大
cloudra/tools/core/gaussian_frames.py:314       for path_no in (1, 2):
cloudra/tools/core/gaussian_frames.py:315           side = path_rows[path_no]
cloudra/tools/core/gaussian_frames.py:316           if side and side[0][1] > 0:
cloudra/tools/core/gaussian_frames.py:317               pts = pts1 if path_no == 1 else pts2
cloudra/tools/core/gaussian_frames.py:318               for i in range(len(side)):
cloudra/tools/core/gaussian_frames.py:319                   pt = pts[i] if i < len(pts) else None
cloudra/tools/core/gaussian_frames.py:320                   fr = _make_frame(idx, path_no, pt or {"point": i + 1, "coords":
     None}, side[i])
cloudra/tools/core/gaussian_frames.py:321                   if i == len(side) - 1:
cloudra/tools/core/gaussian_frames.py:322                       fr.converged = conv[path_no]
cloudra/tools/core/gaussian_frames.py:323                   frames.append(fr)
cloudra/tools/core/gaussian_frames.py:324                   idx += 1
cloudra/tools/core/gaussian_frames.py:325
cloudra/tools/core/gaussian_frames.py:326       rx_values = [f.reaction_coordinate for f in frames if f.reaction_coordinate
     is not None]
cloudra/tools/core/gaussian_frames.py:327       meta = {
cloudra/tools/core/gaussian_frames.py:328           "ts_energy_hartree": ts_energy,
cloudra/tools/core/gaussian_frames.py:329           "num_points_forward": len(pts1),
cloudra/tools/core/gaussian_frames.py:330           "num_points_backward": len(pts2),
cloudra/tools/core/gaussian_frames.py:331           "rx_coord_range": (min(rx_values), max(rx_values)) if rx_values else
    None,
cloudra/tools/core/gaussian_frames.py:332           "endpoint_energies_hartree": (
cloudra/tools/core/gaussian_frames.py:333               frames[0].scf_energy_hartree,
cloudra/tools/core/gaussian_frames.py:334               frames[-1].scf_energy_hartree,
cloudra/tools/core/gaussian_frames.py:335           ) if frames else None,
cloudra/tools/core/gaussian_frames.py:336           "forward_converged": conv[1],
```

```
cloudra/tools/core/gaussian_frames.py:337           "backward_converged": conv[2],
cloudra/tools/core/gaussian_frames.py:338       }
cloudra/tools/core/gaussian_frames.py:339       return frames, meta
cloudra/tools/core/gaussian_frames.py:340
cloudra/tools/core/gaussian_frames.py:341
cloudra/tools/core/gaussian_frames.py:342   def extract_irc(log_path: str):
cloudra/tools/core/gaussian_frames.py:343       """解析 IRC 日志，返回 (list[FrameRecord], meta dict)。供 log_parser 汇总用。"""
cloudra/tools/core/gaussian_frames.py:344       return _parse_irc(_read_text(log_path))
cloudra/tools/core/gaussian_frames.py:345
cloudra/tools/core/gaussian_frames.py:346
cloudra/tools/core/gaussian_frames.py:347   # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:348   # cclib 分支：scan / opt / freq / sp
cloudra/tools/core/gaussian_frames.py:349   # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:350
cloudra/tools/core/gaussian_frames.py:351   def _ccread(log_path: str):
cloudra/tools/core/gaussian_frames.py:352       import cclib
cloudra/tools/core/gaussian_frames.py:353
cloudra/tools/core/gaussian_frames.py:354       data = cclib.io.ccread(log_path)
cloudra/tools/core/gaussian_frames.py:355       if data is None:
cloudra/tools/core/gaussian_frames.py:356           raise ValueError(f"cclib 未能解析日志: {log_path}")
cloudra/tools/core/gaussian_frames.py:357       return data
cloudra/tools/core/gaussian_frames.py:358
cloudra/tools/core/gaussian_frames.py:359
cloudra/tools/core/gaussian_frames.py:360   def _as_float_list(values) -> list[float]:
cloudra/tools/core/gaussian_frames.py:361       import numpy as np
cloudra/tools/core/gaussian_frames.py:362
cloudra/tools/core/gaussian_frames.py:363       return [float(v) for v in np.asarray(list(values), dtype=float)]
cloudra/tools/core/gaussian_frames.py:364
cloudra/tools/core/gaussian_frames.py:365
cloudra/tools/core/gaussian_frames.py:366   def _extract_scan_frames(log_path: str) -> list[FrameRecord]:
cloudra/tools/core/gaussian_frames.py:367       data = _ccread(log_path)
cloudra/tools/core/gaussian_frames.py:368       energies = getattr(data, "scanenergies", None)
cloudra/tools/core/gaussian_frames.py:369       parm = getattr(data, "scanparm", None)
cloudra/tools/core/gaussian_frames.py:370       if energies is None or parm is None or len(parm) == 0:
cloudra/tools/core/gaussian_frames.py:371           return []
cloudra/tools/core/gaussian_frames.py:372       energies_ev = _as_float_list(energies)
cloudra/tools/core/gaussian_frames.py:373       parm_values = _as_float_list(parm[0])  # 多参数扫描 v1 只取第一个参数
cloudra/tools/core/gaussian_frames.py:374       coords = getattr(data, "scancoords", None)
cloudra/tools/core/gaussian_frames.py:375       atomnos = getattr(data, "atomnos", None)
cloudra/tools/core/gaussian_frames.py:376       n = len(energies_ev)
cloudra/tools/core/gaussian_frames.py:377       has_coords = coords is not None and atomnos is not None and len(coords) ==
n
cloudra/tools/core/gaussian_frames.py:378
cloudra/tools/core/gaussian_frames.py:379       frames = []
cloudra/tools/core/gaussian_frames.py:380       for i in range(n):
cloudra/tools/core/gaussian_frames.py:381           rx = parm_values[i] if i < len(parm_values) else None
cloudra/tools/core/gaussian_frames.py:382           xyz = ""
cloudra/tools/core/gaussian_frames.py:383           if has_coords:
cloudra/tools/core/gaussian_frames.py:384               xyz = _coords_to_xyz(atomnos, coords[i], comment=f"scan point={i}
parm={rx}")
cloudra/tools/core/gaussian_frames.py:385           frames.append(FrameRecord(
cloudra/tools/core/gaussian_frames.py:386               frame=i,
cloudra/tools/core/gaussian_frames.py:387               reaction_coordinate=rx,
cloudra/tools/core/gaussian_frames.py:388               scf_energy_hartree=energies_ev[i] / EV_PER_HARTREE,
```

```
cloudra/tools/core/gaussian_frames.py:389              xyz=xyz,
cloudra/tools/core/gaussian_frames.py:390          ))
cloudra/tools/core/gaussian_frames.py:391      return frames
cloudra/tools/core/gaussian_frames.py:392
cloudra/tools/core/gaussian_frames.py:393
cloudra/tools/core/gaussian_frames.py:394  def _opt_converged(data) -> bool | None:
cloudra/tools/core/gaussian_frames.py:395      optdone = getattr(data, "optdone", None)
cloudra/tools/core/gaussian_frames.py:396      if isinstance(optdone, bool):
cloudra/tools/core/gaussian_frames.py:397          return optdone
cloudra/tools/core/gaussian_frames.py:398      if optdone:  # 部分 cclib 版本中 optdone 是收敛步索引列表
cloudra/tools/core/gaussian_frames.py:399          return True
cloudra/tools/core/gaussian_frames.py:400      return None
cloudra/tools/core/gaussian_frames.py:401
cloudra/tools/core/gaussian_frames.py:402
cloudra/tools/core/gaussian_frames.py:403  def _get_vibfreqs(data) -> list[float]:
cloudra/tools/core/gaussian_frames.py:404      vibfreqs = getattr(data, "vibfreqs", None)
cloudra/tools/core/gaussian_frames.py:405      if vibfreqs is None or len(vibfreqs) == 0:
cloudra/tools/core/gaussian_frames.py:406          return []
cloudra/tools/core/gaussian_frames.py:407      return _as_float_list(vibfreqs)
cloudra/tools/core/gaussian_frames.py:408
cloudra/tools/core/gaussian_frames.py:409
cloudra/tools/core/gaussian_frames.py:410  def _extract_opt_frames(log_path: str) -> list[FrameRecord]:
cloudra/tools/core/gaussian_frames.py:411      data = _ccread(log_path)
cloudra/tools/core/gaussian_frames.py:412      energies = getattr(data, "scfenergies", None)
cloudra/tools/core/gaussian_frames.py:413      atomcoords = getattr(data, "atomcoords", None)
cloudra/tools/core/gaussian_frames.py:414      atomnos = getattr(data, "atomnos", None)
cloudra/tools/core/gaussian_frames.py:415      if energies is None or atomcoords is None or atomnos is None:
cloudra/tools/core/gaussian_frames.py:416          return []
cloudra/tools/core/gaussian_frames.py:417      energies_ev = _as_float_list(energies)
cloudra/tools/core/gaussian_frames.py:418      n = min(len(energies_ev), len(atomcoords))
cloudra/tools/core/gaussian_frames.py:419      converged = _opt_converged(data)
cloudra/tools/core/gaussian_frames.py:420      freqs = _get_vibfreqs(data)
cloudra/tools/core/gaussian_frames.py:421
cloudra/tools/core/gaussian_frames.py:422      frames = []
cloudra/tools/core/gaussian_frames.py:423      for i in range(n):
cloudra/tools/core/gaussian_frames.py:424          is_last = i == n - 1
cloudra/tools/core/gaussian_frames.py:425          frames.append(FrameRecord(
cloudra/tools/core/gaussian_frames.py:426              frame=i,
cloudra/tools/core/gaussian_frames.py:427              scf_energy_hartree=energies_ev[i] / EV_PER_HARTREE,
cloudra/tools/core/gaussian_frames.py:428              frequencies_cm=freqs if (is_last and freqs) else [],
cloudra/tools/core/gaussian_frames.py:429              xyz=_coords_to_xyz(atomnos, atomcoords[i], comment=f"opt step={i}")
,
cloudra/tools/core/gaussian_frames.py:430              converged=converged if is_last else None,
cloudra/tools/core/gaussian_frames.py:431          ))
cloudra/tools/core/gaussian_frames.py:432      return frames
cloudra/tools/core/gaussian_frames.py:433
cloudra/tools/core/gaussian_frames.py:434
cloudra/tools/core/gaussian_frames.py:435  def _extract_sp_frames(log_path: str) -> list[FrameRecord]:
cloudra/tools/core/gaussian_frames.py:436      data = _ccread(log_path)
cloudra/tools/core/gaussian_frames.py:437      energies = getattr(data, "scfenergies", None)
cloudra/tools/core/gaussian_frames.py:438      atomcoords = getattr(data, "atomcoords", None)
cloudra/tools/core/gaussian_frames.py:439      atomnos = getattr(data, "atomnos", None)
cloudra/tools/core/gaussian_frames.py:440      if atomcoords is None or atomnos is None or len(atomcoords) == 0:
```

```
cloudra/tools/core/gaussian_frames.py:441           return []
cloudra/tools/core/gaussian_frames.py:442       energy_hartree = None
cloudra/tools/core/gaussian_frames.py:443       if energies is not None and len(energies) > 0:
cloudra/tools/core/gaussian_frames.py:444           energy_hartree = _as_float_list(energies)[-1] / EV_PER_HARTREE
cloudra/tools/core/gaussian_frames.py:445       return [FrameRecord(
cloudra/tools/core/gaussian_frames.py:446           frame=0,
cloudra/tools/core/gaussian_frames.py:447           scf_energy_hartree=energy_hartree,
cloudra/tools/core/gaussian_frames.py:448           frequencies_cm=_get_vibfreqs(data),
cloudra/tools/core/gaussian_frames.py:449           xyz=_coords_to_xyz(atomnos, atomcoords[-1], comment="single point"),
cloudra/tools/core/gaussian_frames.py:450       )]
cloudra/tools/core/gaussian_frames.py:451
cloudra/tools/core/gaussian_frames.py:452
cloudra/tools/core/gaussian_frames.py:453   # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:454   # 公共入口
cloudra/tools/core/gaussian_frames.py:455   # ---------------------------------------------------------------------------
cloudra/tools/core/gaussian_frames.py:456
cloudra/tools/core/gaussian_frames.py:457   def extract_frames(log_path: str) -> list[FrameRecord]:
cloudra/tools/core/gaussian_frames.py:458       """按识别到的任务类型提取关键帧。无法识别的日志返回空列表。"""
cloudra/tools/core/gaussian_frames.py:459       jobs = detect_job_types(log_path)
cloudra/tools/core/gaussian_frames.py:460       if "irc" in jobs:
cloudra/tools/core/gaussian_frames.py:461           frames, _meta = _parse_irc(_read_text(log_path))
cloudra/tools/core/gaussian_frames.py:462           return frames
cloudra/tools/core/gaussian_frames.py:463       if "scan" in jobs:
cloudra/tools/core/gaussian_frames.py:464           return _extract_scan_frames(log_path)
cloudra/tools/core/gaussian_frames.py:465       if "opt" in jobs or "ts" in jobs:
cloudra/tools/core/gaussian_frames.py:466           return _extract_opt_frames(log_path)
cloudra/tools/core/gaussian_frames.py:467       # freq / sp：单帧
cloudra/tools/core/gaussian_frames.py:468       return _extract_sp_frames(log_path)
cloudra/tools/core/gaussian_frames.py:469
cloudra/tools/core/gaussian_frames.py:470
cloudra/tools/core/gaussian_frames.py:471   def _fmt_float(value: float, digits: int = 12) -> str:
cloudra/tools/core/gaussian_frames.py:472       return f"{value:.{digits}g}"
cloudra/tools/core/gaussian_frames.py:473
cloudra/tools/core/gaussian_frames.py:474
cloudra/tools/core/gaussian_frames.py:475   def frames_to_csv(frames: list[FrameRecord], output_path: str) -> str:
cloudra/tools/core/gaussian_frames.py:476       """将关键帧写为 CSV（xyz 为多行字段，靠 csv 引号机制，pandas/DictReader 可读）。"""
cloudra/tools/core/gaussian_frames.py:477       columns = [
cloudra/tools/core/gaussian_frames.py:478           "frame", "path_number", "reaction_coordinate",
cloudra/tools/core/gaussian_frames.py:479           "scf_energy_hartree", "scf_energy_ev", "frequencies_cm", "xyz",
cloudra/tools/core/gaussian_frames.py:480       ]
cloudra/tools/core/gaussian_frames.py:481       out_dir = os.path.dirname(output_path)
cloudra/tools/core/gaussian_frames.py:482       if out_dir:
cloudra/tools/core/gaussian_frames.py:483           os.makedirs(out_dir, exist_ok=True)
cloudra/tools/core/gaussian_frames.py:484       with open(output_path, "w", newline="", encoding="utf-8") as f:
cloudra/tools/core/gaussian_frames.py:485           writer = csv.DictWriter(f, fieldnames=columns)
cloudra/tools/core/gaussian_frames.py:486           writer.writeheader()
cloudra/tools/core/gaussian_frames.py:487           for fr in frames:
cloudra/tools/core/gaussian_frames.py:488               hartree = fr.scf_energy_hartree
cloudra/tools/core/gaussian_frames.py:489               writer.writerow({
cloudra/tools/core/gaussian_frames.py:490                   "frame": fr.frame,
cloudra/tools/core/gaussian_frames.py:491                   "path_number": "" if fr.path_number is None else fr.path_number
    ,
cloudra/tools/core/gaussian_frames.py:492                   "reaction_coordinate": "" if fr.reaction_coordinate is None
    else _fmt_float(fr.reaction_coordinate),
```

```
cloudra/tools/core/gaussian_frames.py:493                 "scf_energy_hartree": "" if hartree is None else _fmt_float(hartree),
cloudra/tools/core/gaussian_frames.py:494                 "scf_energy_ev": "" if hartree is None else _fmt_float(hartree * EV_PER_HARTREE),
cloudra/tools/core/gaussian_frames.py:495                 "frequencies_cm": ";".join(_fmt_float(v) for v in fr.frequencies_cm),
cloudra/tools/core/gaussian_frames.py:496                 "xyz": fr.xyz,
cloudra/tools/core/gaussian_frames.py:497             })
cloudra/tools/core/gaussian_frames.py:498     return output_path
cloudra/tools/core/permission_model.py:1  """
cloudra/tools/core/permission_model.py:2  Permission Model for Cloudra Tools.
cloudra/tools/core/permission_model.py:3
cloudra/tools/core/permission_model.py:4  Replaces the fragile list[bool] permission system with a robust IntFlag-based model.
cloudra/tools/core/permission_model.py:5  """
cloudra/tools/core/permission_model.py:6
cloudra/tools/core/permission_model.py:7  from enum import IntFlag
cloudra/tools/core/permission_model.py:8
cloudra/tools/core/permission_model.py:9
cloudra/tools/core/permission_model.py:10 class Permission(IntFlag):
cloudra/tools/core/permission_model.py:11     """Bit-flag permissions for tools and agents."""
cloudra/tools/core/permission_model.py:12     NONE = 0
cloudra/tools/core/permission_model.py:13     READ = 1
cloudra/tools/core/permission_model.py:14     WRITE = 2
cloudra/tools/core/permission_model.py:15     MANAGE = 4
cloudra/tools/core/permission_model.py:16     SCRIPT = 8
cloudra/tools/core/permission_model.py:17     DESTRUCTIVE = 16
cloudra/tools/core/permission_model.py:18
cloudra/tools/core/permission_model.py:19     # Common composites
cloudra/tools/core/permission_model.py:20     ALL = READ | WRITE | MANAGE | SCRIPT | DESTRUCTIVE
cloudra/tools/core/permission_model.py:21
cloudra/tools/core/permission_model.py:22
cloudra/tools/core/permission_model.py:23 def check_permission(agent_perms: Permission, tool_perms: Permission) -> bool:
cloudra/tools/core/permission_model.py:24     """
cloudra/tools/core/permission_model.py:25     Check if an agent has sufficient permissions for a tool.
cloudra/tools/core/permission_model.py:26
cloudra/tools/core/permission_model.py:27     Logic: tool_perms must be a subset of agent_perms.
cloudra/tools/core/permission_model.py:28     i.e. every bit set in tool_perms must also be set in agent_perms.
cloudra/tools/core/permission_model.py:29     """
cloudra/tools/core/permission_model.py:30     if tool_perms == Permission.NONE:
cloudra/tools/core/permission_model.py:31         return True
cloudra/tools/core/permission_model.py:32     return (tool_perms & agent_perms) == tool_perms
cloudra/tools/core/permission_model.py:33
cloudra/tools/core/permission_model.py:34
cloudra/tools/core/permission_model.py:35 def permission_to_dict(perms: Permission) -> dict[str, bool]:
cloudra/tools/core/permission_model.py:36     """Convert Permission flags to a human-readable dict."""
cloudra/tools/core/permission_model.py:37     return {
cloudra/tools/core/permission_model.py:38         'is_read': bool(perms & Permission.READ),
cloudra/tools/core/permission_model.py:39         'is_write': bool(perms & Permission.WRITE),
cloudra/tools/core/permission_model.py:40         'is_manage': bool(perms & Permission.MANAGE),
cloudra/tools/core/permission_model.py:41         'is_script': bool(perms & Permission.SCRIPT),
cloudra/tools/core/permission_model.py:42         'is_destructive': bool(perms & Permission.DESTRUCTIVE),
cloudra/tools/core/permission_model.py:43     }
cloudra/tools/core/workspace_mutation.py:1 """文件变更工具共享的审批、应用与事件流水线。"""
cloudra/tools/core/workspace_mutation.py:2
cloudra/tools/core/workspace_mutation.py:3 from __future__ import annotations
```

```
cloudra/tools/core/workspace_mutation.py:4
cloudra/tools/core/workspace_mutation.py:5  import asyncio
cloudra/tools/core/workspace_mutation.py:6  from pathlib import Path
cloudra/tools/core/workspace_mutation.py:7  from typing import TypeVar
cloudra/tools/core/workspace_mutation.py:8
cloudra/tools/core/workspace_mutation.py:9  from pydantic import Field
cloudra/tools/core/workspace_mutation.py:10
cloudra/tools/core/workspace_mutation.py:11 from cloudra.base.session_class import Session
cloudra/tools/core/workspace_mutation.py:12 from cloudra.base.tools_class import Tool, output_schema
cloudra/tools/core/workspace_mutation.py:13 from cloudra.persistence.workspace import (
cloudra/tools/core/workspace_mutation.py:14     PreparedFileChange,
cloudra/tools/core/workspace_mutation.py:15     WorkspaceConflictError,
cloudra/tools/core/workspace_mutation.py:16     WorkspaceError,
cloudra/tools/core/workspace_mutation.py:17     WorkspaceService,
cloudra/tools/core/workspace_mutation.py:18     turn_diff_tracker,
cloudra/tools/core/workspace_mutation.py:19 )
cloudra/tools/core/workspace_mutation.py:20
cloudra/tools/core/workspace_mutation.py:21
cloudra/tools/core/workspace_mutation.py:22 class WorkspaceMutationOutput(output_schema):
cloudra/tools/core/workspace_mutation.py:23     is_written: bool = Field(description="Whether the file write completed")
cloudra/tools/core/workspace_mutation.py:24     filepath: str = Field(description="File path relative to the working
directory")
cloudra/tools/core/workspace_mutation.py:25     status: str = Field(description="applied, declined, conflict, or failed")
cloudra/tools/core/workspace_mutation.py:26     change_id: str = Field(default="", description="ID of this file change")
cloudra/tools/core/workspace_mutation.py:27     before_sha256: str | None = Field(default=None, description="File hash
before the change")
cloudra/tools/core/workspace_mutation.py:28     after_sha256: str | None = Field(default=None, description="Candidate or
post-write file hash")
cloudra/tools/core/workspace_mutation.py:29     diff: str = Field(default="", description="Unified diff of this change")
cloudra/tools/core/workspace_mutation.py:30     message: str = Field(default="", description="Status or error message")
cloudra/tools/core/workspace_mutation.py:31
cloudra/tools/core/workspace_mutation.py:32
cloudra/tools/core/workspace_mutation.py:33 _OutputT = TypeVar("_OutputT", bound=WorkspaceMutationOutput)
cloudra/tools/core/workspace_mutation.py:34
cloudra/tools/core/workspace_mutation.py:35
cloudra/tools/core/workspace_mutation.py:36 def workspace_for_tool(tool: Tool) -> WorkspaceService:
cloudra/tools/core/workspace_mutation.py:37     root = getattr(tool.subagent, "working_directory", None)
cloudra/tools/core/workspace_mutation.py:38     if not root:
cloudra/tools/core/workspace_mutation.py:39         import os
cloudra/tools/core/workspace_mutation.py:40
cloudra/tools/core/workspace_mutation.py:41         root = os.getcwd()
cloudra/tools/core/workspace_mutation.py:42     return WorkspaceService(root)
cloudra/tools/core/workspace_mutation.py:43
cloudra/tools/core/workspace_mutation.py:44
cloudra/tools/core/workspace_mutation.py:45 def _approval_details(action: str, change: PreparedFileChange) -> str:
cloudra/tools/core/workspace_mutation.py:46     diff_preview = change.diff
cloudra/tools/core/workspace_mutation.py:47     if len(diff_preview) > 12_000:
cloudra/tools/core/workspace_mutation.py:48         diff_preview = diff_preview[:12_000] + "\n... 审批预览已截断，完整 diff 可
在界面查看 ...\n"
cloudra/tools/core/workspace_mutation.py:49     return (
cloudra/tools/core/workspace_mutation.py:50         f"【动作】: {action}\n"
cloudra/tools/core/workspace_mutation.py:51         f"【相对路径】: {change.relative_path}\n"
cloudra/tools/core/workspace_mutation.py:52         f"【变更规模】: +{change.additions} -{change.deletions}\n"
cloudra/tools/core/workspace_mutation.py:53         f"【修改前 SHA256】: {change.before_sha256 or 'missing'}\n"
cloudra/tools/core/workspace_mutation.py:54         f"【修改后 SHA256】: {change.after_sha256}\n"
cloudra/tools/core/workspace_mutation.py:55         f"【Unified Diff】:\n{diff_preview}"
```

```
cloudra/tools/core/workspace_mutation.py:56     )
cloudra/tools/core/workspace_mutation.py:57
cloudra/tools/core/workspace_mutation.py:58
cloudra/tools/core/workspace_mutation.py:59 async def record_preparation_failure(
cloudra/tools/core/workspace_mutation.py:60     tool: Tool,
cloudra/tools/core/workspace_mutation.py:61     session: Session,
cloudra/tools/core/workspace_mutation.py:62     relative_path: str,
cloudra/tools/core/workspace_mutation.py:63     status: str,
cloudra/tools/core/workspace_mutation.py:64     message: str,
cloudra/tools/core/workspace_mutation.py:65 ) -> None:
cloudra/tools/core/workspace_mutation.py:66     await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:67         session,
cloudra/tools/core/workspace_mutation.py:68         "file_change_failed",
cloudra/tools/core/workspace_mutation.py:69         {
cloudra/tools/core/workspace_mutation.py:70             "path": relative_path,
cloudra/tools/core/workspace_mutation.py:71             "status": status,
cloudra/tools/core/workspace_mutation.py:72             "diff": "",
cloudra/tools/core/workspace_mutation.py:73             "message": message,
cloudra/tools/core/workspace_mutation.py:74         },
cloudra/tools/core/workspace_mutation.py:75     )
cloudra/tools/core/workspace_mutation.py:76
cloudra/tools/core/workspace_mutation.py:77
cloudra/tools/core/workspace_mutation.py:78 async def execute_prepared_change(
cloudra/tools/core/workspace_mutation.py:79     tool: Tool,
cloudra/tools/core/workspace_mutation.py:80     service: WorkspaceService,
cloudra/tools/core/workspace_mutation.py:81     change: PreparedFileChange,
cloudra/tools/core/workspace_mutation.py:82     session: Session,
cloudra/tools/core/workspace_mutation.py:83     action: str,
cloudra/tools/core/workspace_mutation.py:84     tool_args: dict,
cloudra/tools/core/workspace_mutation.py:85     output_class: type[_OutputT],
cloudra/tools/core/workspace_mutation.py:86 ) -> _OutputT:
cloudra/tools/core/workspace_mutation.py:87     await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:88         session,
cloudra/tools/core/workspace_mutation.py:89         "file_change_proposed",
cloudra/tools/core/workspace_mutation.py:90         change.event_data("proposed"),
cloudra/tools/core/workspace_mutation.py:91     )
cloudra/tools/core/workspace_mutation.py:92     approved, denial_reason = await tool.ensure_safety(
cloudra/tools/core/workspace_mutation.py:93         session,
cloudra/tools/core/workspace_mutation.py:94         details=_approval_details(action, change),
cloudra/tools/core/workspace_mutation.py:95         tool_args=tool_args,
cloudra/tools/core/workspace_mutation.py:96     )
cloudra/tools/core/workspace_mutation.py:97     if not approved:
cloudra/tools/core/workspace_mutation.py:98         message = denial_reason or (
cloudra/tools/core/workspace_mutation.py:99             "User declined the file change. "
cloudra/tools/core/workspace_mutation.py:100                 "Next: explain to the user why the change was needed, adjust
the content, "
cloudra/tools/core/workspace_mutation.py:101                 "or choose a different approach."
cloudra/tools/core/workspace_mutation.py:102             )
cloudra/tools/core/workspace_mutation.py:103             await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:104                 session,
cloudra/tools/core/workspace_mutation.py:105                 "file_change_declined",
cloudra/tools/core/workspace_mutation.py:106                 change.event_data("declined", message),
cloudra/tools/core/workspace_mutation.py:107             )
```

```
cloudra/tools/core/workspace_mutation.py:108            return output_class(
cloudra/tools/core/workspace_mutation.py:109                is_written=False,
cloudra/tools/core/workspace_mutation.py:110                filepath=change.relative_path,
cloudra/tools/core/workspace_mutation.py:111                status="declined",
cloudra/tools/core/workspace_mutation.py:112                change_id=change.change_id,
cloudra/tools/core/workspace_mutation.py:113                before_sha256=change.before_sha256,
cloudra/tools/core/workspace_mutation.py:114                after_sha256=change.after_sha256,
cloudra/tools/core/workspace_mutation.py:115                diff=change.diff,
cloudra/tools/core/workspace_mutation.py:116                message=message,
cloudra/tools/core/workspace_mutation.py:117            )
cloudra/tools/core/workspace_mutation.py:118
cloudra/tools/core/workspace_mutation.py:119        try:
cloudra/tools/core/workspace_mutation.py:120            await asyncio.to_thread(service.apply, change)
cloudra/tools/core/workspace_mutation.py:121        except WorkspaceConflictError as exc:
cloudra/tools/core/workspace_mutation.py:122            message = str(exc)
cloudra/tools/core/workspace_mutation.py:123            await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:124                session,
cloudra/tools/core/workspace_mutation.py:125                "file_change_failed",
cloudra/tools/core/workspace_mutation.py:126                change.event_data("conflict", message),
cloudra/tools/core/workspace_mutation.py:127            )
cloudra/tools/core/workspace_mutation.py:128            return output_class(
cloudra/tools/core/workspace_mutation.py:129                is_written=False,
cloudra/tools/core/workspace_mutation.py:130                filepath=change.relative_path,
cloudra/tools/core/workspace_mutation.py:131                status="conflict",
cloudra/tools/core/workspace_mutation.py:132                change_id=change.change_id,
cloudra/tools/core/workspace_mutation.py:133                before_sha256=change.before_sha256,
cloudra/tools/core/workspace_mutation.py:134                after_sha256=change.after_sha256,
cloudra/tools/core/workspace_mutation.py:135                diff=change.diff,
cloudra/tools/core/workspace_mutation.py:136                message=message,
cloudra/tools/core/workspace_mutation.py:137            )
cloudra/tools/core/workspace_mutation.py:138        except (OSError, WorkspaceError) as exc:
cloudra/tools/core/workspace_mutation.py:139            message = str(exc)
cloudra/tools/core/workspace_mutation.py:140            await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:141                session,
cloudra/tools/core/workspace_mutation.py:142                "file_change_failed",
cloudra/tools/core/workspace_mutation.py:143                change.event_data("failed", message),
cloudra/tools/core/workspace_mutation.py:144            )
cloudra/tools/core/workspace_mutation.py:145            return output_class(
cloudra/tools/core/workspace_mutation.py:146                is_written=False,
cloudra/tools/core/workspace_mutation.py:147                filepath=change.relative_path,
cloudra/tools/core/workspace_mutation.py:148                status="failed",
cloudra/tools/core/workspace_mutation.py:149                change_id=change.change_id,
cloudra/tools/core/workspace_mutation.py:150                before_sha256=change.before_sha256,
cloudra/tools/core/workspace_mutation.py:151                after_sha256=change.after_sha256,
cloudra/tools/core/workspace_mutation.py:152                diff=change.diff,
cloudra/tools/core/workspace_mutation.py:153                message=message,
cloudra/tools/core/workspace_mutation.py:154            )
cloudra/tools/core/workspace_mutation.py:155
cloudra/tools/core/workspace_mutation.py:156        await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:157            session,
cloudra/tools/core/workspace_mutation.py:158            "file_change_applied",
cloudra/tools/core/workspace_mutation.py:159            change.event_data("applied"),
```

```
cloudra/tools/core/workspace_mutation.py:160        )
cloudra/tools/core/workspace_mutation.py:161        conversation_id = getattr(tool.subagent, "conversation_id", "default")
cloudra/tools/core/workspace_mutation.py:162        from cloudra.persistence.database import ConversationDB
cloudra/tools/core/workspace_mutation.py:163
cloudra/tools/core/workspace_mutation.py:164        if Path(ConversationDB(conversation_id).path).is_file():
cloudra/tools/core/workspace_mutation.py:165            try:
cloudra/tools/core/workspace_mutation.py:166                from cloudra.persistence.artifacts import ArtifactService
cloudra/tools/core/workspace_mutation.py:167
cloudra/tools/core/workspace_mutation.py:168                await asyncio.to_thread(
cloudra/tools/core/workspace_mutation.py:169                    ArtifactService().register_file,
cloudra/tools/core/workspace_mutation.py:170                    conversation_id=conversation_id,
cloudra/tools/core/workspace_mutation.py:171                    session_id=session.id,
cloudra/tools/core/workspace_mutation.py:172                    source=change.absolute_path,
cloudra/tools/core/workspace_mutation.py:173                    allowed_root=service.root,
cloudra/tools/core/workspace_mutation.py:174                    name=change.relative_path,
cloudra/tools/core/workspace_mutation.py:175                )
cloudra/tools/core/workspace_mutation.py:176            except (OSError, ValueError) as exc:
cloudra/tools/core/workspace_mutation.py:177                await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:178                    session,
cloudra/tools/core/workspace_mutation.py:179                    "artifact_registration_failed",
cloudra/tools/core/workspace_mutation.py:180                    {
cloudra/tools/core/workspace_mutation.py:181                        "path": change.relative_path,
cloudra/tools/core/workspace_mutation.py:182                        "message": str(exc),
cloudra/tools/core/workspace_mutation.py:183                    },
cloudra/tools/core/workspace_mutation.py:184                )
cloudra/tools/core/workspace_mutation.py:185        session_key = f"{conversation_id}:{session.id}"
cloudra/tools/core/workspace_mutation.py:186        aggregate_diff = turn_diff_tracker.record_applied(session_key, service,
     change)
cloudra/tools/core/workspace_mutation.py:187        await tool.record_session_event(
cloudra/tools/core/workspace_mutation.py:188            session,
cloudra/tools/core/workspace_mutation.py:189            "turn_diff_updated",
cloudra/tools/core/workspace_mutation.py:190            {
cloudra/tools/core/workspace_mutation.py:191                "change_id": change.change_id,
cloudra/tools/core/workspace_mutation.py:192                "path": change.relative_path,
cloudra/tools/core/workspace_mutation.py:193                "status": "updated",
cloudra/tools/core/workspace_mutation.py:194                "diff": aggregate_diff,
cloudra/tools/core/workspace_mutation.py:195            },
cloudra/tools/core/workspace_mutation.py:196        )
cloudra/tools/core/workspace_mutation.py:197        return output_class(
cloudra/tools/core/workspace_mutation.py:198            is_written=True,
cloudra/tools/core/workspace_mutation.py:199            filepath=change.relative_path,
cloudra/tools/core/workspace_mutation.py:200            status="applied",
cloudra/tools/core/workspace_mutation.py:201            change_id=change.change_id,
cloudra/tools/core/workspace_mutation.py:202            before_sha256=change.before_sha256,
cloudra/tools/core/workspace_mutation.py:203            after_sha256=change.after_sha256,
cloudra/tools/core/workspace_mutation.py:204            diff=change.diff,
cloudra/tools/core/workspace_mutation.py:205            message="File change written atomically",
cloudra/tools/core/workspace_mutation.py:206        )
cloudra/tools/registry.py:1 """Explicit built-in tool manifest.
cloudra/tools/registry.py:2
cloudra/tools/registry.py:3 Keep this module dependency-free: :mod:`cloudra.core.agent` imports
cloudra/tools/registry.py:4 ``cloudra.tools`` while several tool implementations import ``cloudra.core.agent``.
cloudra/tools/registry.py:5 The registry therefore stores import paths instead of importing tool classes at
```

```
cloudra/tools/registry.py:6 module import time.
cloudra/tools/registry.py:7
cloudra/tools/registry.py:8 Optional semantic fields are the single source of truth for downstream policy
cloudra/tools/registry.py:9 projections (DAG gate / runtime profiles / evidence derivation).  Downstream
cloudra/tools/registry.py:10    modules must query :mod:`cloudra.tools.capabilities` instead of maintaining
cloudra/tools/registry.py:11    their own per-tool name sets.
cloudra/tools/registry.py:12    """
cloudra/tools/registry.py:13
cloudra/tools/registry.py:14    from dataclasses import dataclass
cloudra/tools/registry.py:15
cloudra/tools/registry.py:16
cloudra/tools/registry.py:17    @dataclass(frozen=True)
cloudra/tools/registry.py:18    class ToolRegistration:
cloudra/tools/registry.py:19        module: str
cloudra/tools/registry.py:20        class_name: str
cloudra/tools/registry.py:21
cloudra/tools/registry.py:22        # Declared wire name. Only needed when the tool has non-default semantics
cloudra/tools/registry.py:23        # (or a tool_name that differs from class_name); capabilities only projects
cloudra/tools/registry.py:24        # entries that carry it.
cloudra/tools/registry.py:25        tool_name: str = ""
cloudra/tools/registry.py:26
cloudra/tools/registry.py:27        # DAG semantics. All flags default to the conservative/plain behavior.
cloudra/tools/registry.py:28        dag_substantive: bool = False
cloudra/tools/registry.py:29        substantive_predicate: str = ""  # "module:function(arguments)->bool", overrides
dag_substantive
cloudra/tools/registry.py:30        dag_observe: bool = True
cloudra/tools/registry.py:31        supervisor_evidence: bool = False
cloudra/tools/registry.py:32        evidence_deriver: str = ""  # "module:function(current, role, arguments, result)->dict"
cloudra/tools/registry.py:33        # D5：工具结束是否入 CallReview 收件箱（并唤醒 Supervisor 复核）。
cloudra/tools/registry.py:34        dag_review: bool = False
cloudra/tools/registry.py:35        # Phase 3 门槛：mechanism 未被接受前拦截的计算入口（开算/写计算输入）。
cloudra/tools/registry.py:36        dag_compute_entry: bool = False
cloudra/tools/registry.py:37        dag_compute_predicate: str = ""  # "module:function(arguments)->bool", overrides
dag_compute_entry
cloudra/tools/registry.py:38
cloudra/tools/registry.py:39        # Runtime availability. The default keeps the Android allow-list opt-in.
cloudra/tools/registry.py:40        runtime_profiles: frozenset[str] = frozenset({"desktop"})
cloudra/tools/registry.py:41
cloudra/tools/registry.py:42
cloudra/tools/registry.py:43    TOOL_REGISTRY: tuple[ToolRegistration, ...] = (
cloudra/tools/registry.py:44        ToolRegistration(
cloudra/tools/registry.py:45            module='cloudra.tools.Bashtool.bashtools',
cloudra/tools/registry.py:46            class_name='Bashtools',
cloudra/tools/registry.py:47            tool_name='bashtools',
cloudra/tools/registry.py:48            substantive_predicate='cloudra.tools.core.bash_habit_guard:
is_substantive_bash_command',
cloudra/tools/registry.py:49            dag_compute_predicate='cloudra.tools.core.bash_habit_guard:
is_substantive_bash_command',
cloudra/tools/registry.py:50        ),
cloudra/tools/registry.py:51        ToolRegistration('cloudra.tools.Browsertool.headless_browser', 'HeadlessBrowser'),
cloudra/tools/registry.py:52        ToolRegistration(
cloudra/tools/registry.py:53            module='cloudra.tools.Browsertool.web_fetcher',
cloudra/tools/registry.py:54            class_name='WebFetcher',
cloudra/tools/registry.py:55            tool_name='web_fetcher',
cloudra/tools/registry.py:56            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:57        ),
```

```
cloudra/tools/registry.py:58        ToolRegistration(
cloudra/tools/registry.py:59            module='cloudra.tools.Chemtool.conformer_search',
cloudra/tools/registry.py:60            class_name='ConformerSearch',
cloudra/tools/registry.py:61            tool_name='conformer_search',
cloudra/tools/registry.py:62            dag_substantive=True,
cloudra/tools/registry.py:63            dag_compute_entry=True,
cloudra/tools/registry.py:64        ),
cloudra/tools/registry.py:65        ToolRegistration(
cloudra/tools/registry.py:66            module='cloudra.tools.Chemtool.electronic_analysis',
cloudra/tools/registry.py:67            class_name='ElectronicAnalysis',
cloudra/tools/registry.py:68            tool_name='electronic_analysis',
cloudra/tools/registry.py:69            dag_substantive=True,
cloudra/tools/registry.py:70            dag_compute_entry=True,
cloudra/tools/registry.py:71        ),
cloudra/tools/registry.py:72        ToolRegistration('cloudra.tools.Chemtool.format_converter', 'FormatConverter'),
cloudra/tools/registry.py:73        ToolRegistration(
cloudra/tools/registry.py:74            module='cloudra.tools.Chemtool.gjf_writer',
cloudra/tools/registry.py:75            class_name='GjfWriter',
cloudra/tools/registry.py:76            tool_name='gjf_writer',
cloudra/tools/registry.py:77            dag_substantive=True,
cloudra/tools/registry.py:78            dag_compute_entry=True,
cloudra/tools/registry.py:79        ),
cloudra/tools/registry.py:80        ToolRegistration('cloudra.tools.Chemtool.image_to_smiles', 'ImageToSmiles'),
cloudra/tools/registry.py:81        ToolRegistration('cloudra.tools.Chemtool.internal_coordinates', 'InternalCoordinates'),
cloudra/tools/registry.py:82        ToolRegistration(
cloudra/tools/registry.py:83            module='cloudra.tools.Chemtool.ir_processor',
cloudra/tools/registry.py:84            class_name='IRProcessor',
cloudra/tools/registry.py:85            tool_name='ir_processor',
cloudra/tools/registry.py:86            dag_substantive=True,
cloudra/tools/registry.py:87        ),
cloudra/tools/registry.py:88        ToolRegistration(
cloudra/tools/registry.py:89            module='cloudra.tools.Chemtool.log_parser',
cloudra/tools/registry.py:90            class_name='LogParser',
cloudra/tools/registry.py:91            tool_name='gaussian_log_parser',
cloudra/tools/registry.py:92            dag_substantive=True,
cloudra/tools/registry.py:93            supervisor_evidence=True,
cloudra/tools/registry.py:94            evidence_deriver='cloudra.tools.Chemtool.log_parser:derive_delivery_changes',
cloudra/tools/registry.py:95            dag_review=True,
cloudra/tools/registry.py:96        ),
cloudra/tools/registry.py:97        ToolRegistration('cloudra.tools.Chemtool.molecule_drawer', 'MoleculeDrawer'),
cloudra/tools/registry.py:98        ToolRegistration('cloudra.tools.Chemtool.name_to_structure', 'NameToStructure'),
cloudra/tools/registry.py:99        ToolRegistration(
cloudra/tools/registry.py:100           module='cloudra.tools.Chemtool.nmr_processor',
cloudra/tools/registry.py:101           class_name='NMRProcessor',
cloudra/tools/registry.py:102           tool_name='nmr_processor',
cloudra/tools/registry.py:103           dag_substantive=True,
cloudra/tools/registry.py:104       ),
cloudra/tools/registry.py:105       ToolRegistration('cloudra.tools.Chemtool.query_compute_job', 'QueryComputeJob'),
cloudra/tools/registry.py:106       ToolRegistration(
cloudra/tools/registry.py:107           module='cloudra.tools.Chemtool.reaction_mapping',
cloudra/tools/registry.py:108           class_name='ReactionMapping',
cloudra/tools/registry.py:109           tool_name='reaction_mapping',
```

```
cloudra/tools/registry.py:110           evidence_deriver='cloudra.tools.Chemtool.reaction_mapping:derive_delivery_changes',
cloudra/tools/registry.py:111           dag_review=True,
cloudra/tools/registry.py:112       ),
cloudra/tools/registry.py:113       ToolRegistration('cloudra.tools.Chemtool.smiles_to_mol', 'SmilesToMol'),
cloudra/tools/registry.py:114       ToolRegistration(
cloudra/tools/registry.py:115           module='cloudra.tools.Chemtool.submit_compute_job',
cloudra/tools/registry.py:116           class_name='SubmitComputeJob',
cloudra/tools/registry.py:117           tool_name='submit_compute_job',
cloudra/tools/registry.py:118           dag_substantive=True,
cloudra/tools/registry.py:119           dag_compute_entry=True,
cloudra/tools/registry.py:120       ),
cloudra/tools/registry.py:121       ToolRegistration(
cloudra/tools/registry.py:122           module='cloudra.tools.Chemtool.thermo_analysis',
cloudra/tools/registry.py:123           class_name='ThermoAnalysis',
cloudra/tools/registry.py:124           tool_name='thermo_analysis',
cloudra/tools/registry.py:125           dag_substantive=True,
cloudra/tools/registry.py:126           evidence_deriver='cloudra.tools.Chemtool.thermo_analysis:derive_delivery_changes',
cloudra/tools/registry.py:127           dag_review=True,
cloudra/tools/registry.py:128       ),
cloudra/tools/registry.py:129       ToolRegistration(
cloudra/tools/registry.py:130           module='cloudra.tools.Chemtool.ts_guess',
cloudra/tools/registry.py:131           class_name='TsGuess',
cloudra/tools/registry.py:132           tool_name='ts_guess',
cloudra/tools/registry.py:133           dag_substantive=True,
cloudra/tools/registry.py:134           evidence_deriver='cloudra.tools.Chemtool.ts_guess:derive_delivery_changes',
cloudra/tools/registry.py:135           dag_review=True,
cloudra/tools/registry.py:136           dag_compute_predicate='cloudra.tools.Chemtool.ts_guess:is_compute_entry_call',
cloudra/tools/registry.py:137       ),
cloudra/tools/registry.py:138       ToolRegistration('cloudra.tools.Interactool.ask_user', 'AskUser'),
cloudra/tools/registry.py:139       ToolRegistration('cloudra.tools.Interactool.email_reader', 'EmailReader'),
cloudra/tools/registry.py:140       ToolRegistration('cloudra.tools.Interactool.email_sender', 'EmailSender'),
cloudra/tools/registry.py:141       ToolRegistration(
cloudra/tools/registry.py:142           module='cloudra.tools.Managetool.agent_hub',
cloudra/tools/registry.py:143           class_name='AgentHub',
cloudra/tools/registry.py:144           tool_name='agent_hub',
cloudra/tools/registry.py:145           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:146       ),
cloudra/tools/registry.py:147       ToolRegistration(
cloudra/tools/registry.py:148           module='cloudra.tools.Managetool.get_subagent_report',
cloudra/tools/registry.py:149           class_name='GetSubAgentReport',
cloudra/tools/registry.py:150           tool_name='get_subagent_report',
cloudra/tools/registry.py:151           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:152       ),
cloudra/tools/registry.py:153       ToolRegistration(
cloudra/tools/registry.py:154           module='cloudra.tools.Managetool.schedule',
cloudra/tools/registry.py:155           class_name='Schedule',
cloudra/tools/registry.py:156           tool_name='schedule',
cloudra/tools/registry.py:157           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:158       ),
cloudra/tools/registry.py:159       ToolRegistration(
cloudra/tools/registry.py:160           module='cloudra.tools.Managetool.subagent_launcher',
cloudra/tools/registry.py:161           class_name='SubAgentLauncher',
```

```
cloudra/tools/registry.py:162           tool_name='SubAgentLauncher',
cloudra/tools/registry.py:163           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:164           dag_substantive=True,
cloudra/tools/registry.py:165       ),
cloudra/tools/registry.py:166       ToolRegistration(
cloudra/tools/registry.py:167           module='cloudra.tools.Managetool.subagent_swarm',
cloudra/tools/registry.py:168           class_name='SubAgentSwarm',
cloudra/tools/registry.py:169           tool_name='SubAgentSwarm',
cloudra/tools/registry.py:170           dag_substantive=True,
cloudra/tools/registry.py:171       ),
cloudra/tools/registry.py:172       ToolRegistration('cloudra.tools.Plantool.goal', 'GoalSet'),
cloudra/tools/registry.py:173       ToolRegistration(
cloudra/tools/registry.py:174           module='cloudra.tools.Plantool.dag_tools.dag_view',
cloudra/tools/registry.py:175           class_name='DagView',
cloudra/tools/registry.py:176           tool_name='dag_view',
cloudra/tools/registry.py:177           dag_observe=False,
cloudra/tools/registry.py:178       ),
cloudra/tools/registry.py:179       ToolRegistration(
cloudra/tools/registry.py:180           module='cloudra.tools.Plantool.dag_tools.dag_delivery_report',
cloudra/tools/registry.py:181           class_name='DagDeliveryReport',
cloudra/tools/registry.py:182           tool_name='dag_delivery_report',
cloudra/tools/registry.py:183           dag_observe=False,
cloudra/tools/registry.py:184       ),
cloudra/tools/registry.py:185       ToolRegistration(
cloudra/tools/registry.py:186           module='cloudra.tools.Plantool.dag_tools.dag_add_species',
cloudra/tools/registry.py:187           class_name='DagAddSpecies',
cloudra/tools/registry.py:188           tool_name='dag_add_species',
cloudra/tools/registry.py:189           dag_observe=False,
cloudra/tools/registry.py:190       ),
cloudra/tools/registry.py:191       ToolRegistration(
cloudra/tools/registry.py:192           module='cloudra.tools.Plantool.dag_tools.dag_add_step',
cloudra/tools/registry.py:193           class_name='DagAddStep',
cloudra/tools/registry.py:194           tool_name='dag_add_step',
cloudra/tools/registry.py:195           dag_observe=False,
cloudra/tools/registry.py:196       ),
cloudra/tools/registry.py:197       ToolRegistration(
cloudra/tools/registry.py:198           module='cloudra.tools.Plantool.dag_tools.dag_update_delivery_element',
cloudra/tools/registry.py:199           class_name='DagUpdateDeliveryElement',
cloudra/tools/registry.py:200           tool_name='dag_update_delivery_element',
cloudra/tools/registry.py:201           dag_observe=False,
cloudra/tools/registry.py:202       ),
cloudra/tools/registry.py:203       ToolRegistration(
cloudra/tools/registry.py:204           module='cloudra.tools.Plantool.dag_tools.dag_remove_edge',
cloudra/tools/registry.py:205           class_name='DagRemoveEdge',
cloudra/tools/registry.py:206           tool_name='dag_remove_edge',
cloudra/tools/registry.py:207           dag_observe=False,
cloudra/tools/registry.py:208       ),
cloudra/tools/registry.py:209       ToolRegistration(
cloudra/tools/registry.py:210           module='cloudra.tools.Plantool.dag_tools.dag_bind_evidence',
cloudra/tools/registry.py:211           class_name='DagBindEvidence',
cloudra/tools/registry.py:212           tool_name='dag_bind_evidence',
cloudra/tools/registry.py:213           dag_observe=False,
```

```
cloudra/tools/registry.py:214       ),
cloudra/tools/registry.py:215       ToolRegistration(
cloudra/tools/registry.py:216           module='cloudra.tools.Plantool.dag_tools.dag_propose_mechanism',
cloudra/tools/registry.py:217           class_name='DagProposeMechanism',
cloudra/tools/registry.py:218           tool_name='dag_propose_mechanism',
cloudra/tools/registry.py:219           dag_observe=False,
cloudra/tools/registry.py:220       ),
cloudra/tools/registry.py:221       ToolRegistration(
cloudra/tools/registry.py:222           module='cloudra.tools.Plantool.dag_tools.dag_accept_mechanism',
cloudra/tools/registry.py:223           class_name='DagAcceptMechanism',
cloudra/tools/registry.py:224           tool_name='dag_accept_mechanism',
cloudra/tools/registry.py:225           dag_observe=False,
cloudra/tools/registry.py:226       ),
cloudra/tools/registry.py:227       ToolRegistration(
cloudra/tools/registry.py:228           module='cloudra.tools.Plantool.dag_tools.dag_review_call',
cloudra/tools/registry.py:229           class_name='DagReviewCall',
cloudra/tools/registry.py:230           tool_name='dag_review_call',
cloudra/tools/registry.py:231           dag_observe=False,
cloudra/tools/registry.py:232       ),
cloudra/tools/registry.py:233       ToolRegistration(
cloudra/tools/registry.py:234           module='cloudra.tools.Plantool.dag_tools.dag_set_coverage_contract',
cloudra/tools/registry.py:235           class_name='DagSetCoverageContract',
cloudra/tools/registry.py:236           tool_name='dag_set_coverage_contract',
cloudra/tools/registry.py:237           dag_observe=False,
cloudra/tools/registry.py:238       ),
cloudra/tools/registry.py:239       ToolRegistration(
cloudra/tools/registry.py:240           module='cloudra.tools.Plantool.dag_tools.dag_message_supervisor',
cloudra/tools/registry.py:241           class_name='DagMessageSupervisor',
cloudra/tools/registry.py:242           tool_name='dag_message_supervisor',
cloudra/tools/registry.py:243           dag_observe=False,
cloudra/tools/registry.py:244       ),
cloudra/tools/registry.py:245       ToolRegistration('cloudra.tools.Plantool.todo', 'TodoWrite'),
cloudra/tools/registry.py:246       ToolRegistration('cloudra.tools.Plantool.todo_read', 'TodoRead'),
cloudra/tools/registry.py:247       ToolRegistration(
cloudra/tools/registry.py:248           module='cloudra.tools.Searchtool.openalex_searcher',
cloudra/tools/registry.py:249           class_name='OpenAlexSearcher',
cloudra/tools/registry.py:250           tool_name='openalex_searcher',
cloudra/tools/registry.py:251           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:252       ),
cloudra/tools/registry.py:253       ToolRegistration(
cloudra/tools/registry.py:254           module='cloudra.tools.Searchtool.image_reader',
cloudra/tools/registry.py:255           class_name='ImageReader',
cloudra/tools/registry.py:256           tool_name='image_reader',
cloudra/tools/registry.py:257           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:258       ),
cloudra/tools/registry.py:259       ToolRegistration(
cloudra/tools/registry.py:260           module='cloudra.tools.Searchtool.paper_searcher',
cloudra/tools/registry.py:261           class_name='PaperSearcher',
cloudra/tools/registry.py:262           tool_name='paper_searcher',
cloudra/tools/registry.py:263           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:264       ),
cloudra/tools/registry.py:265       ToolRegistration('cloudra.tools.Searchtool.pdf2image', 'pdf2image'),
```

```
cloudra/tools/registry.py:266       ToolRegistration('cloudra.tools.Searchtool.pdf2md', 'pdf2md'),
cloudra/tools/registry.py:267       ToolRegistration(
cloudra/tools/registry.py:268           module='cloudra.tools.Searchtool.text_reader',
cloudra/tools/registry.py:269           class_name='TextReader',
cloudra/tools/registry.py:270           tool_name='text_reader',
cloudra/tools/registry.py:271           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:272       ),
cloudra/tools/registry.py:273       ToolRegistration(
cloudra/tools/registry.py:274           module='cloudra.tools.Searchtool.web_search',
cloudra/tools/registry.py:275           class_name='WebSearch',
cloudra/tools/registry.py:276           tool_name='web_search',
cloudra/tools/registry.py:277           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:278       ),
cloudra/tools/registry.py:279       ToolRegistration(
cloudra/tools/registry.py:280           module='cloudra.tools.Skilltool.get_skill',
cloudra/tools/registry.py:281           class_name='GetSkill',
cloudra/tools/registry.py:282           tool_name='getskill',
cloudra/tools/registry.py:283           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:284           dag_observe=False,
cloudra/tools/registry.py:285       ),
cloudra/tools/registry.py:286       ToolRegistration(
cloudra/tools/registry.py:287           module='cloudra.tools.Stoptool.stop_tool',
cloudra/tools/registry.py:288           class_name='StopTool',
cloudra/tools/registry.py:289           tool_name='stop',
cloudra/tools/registry.py:290           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:291       ),
cloudra/tools/registry.py:292       ToolRegistration(
cloudra/tools/registry.py:293           module='cloudra.tools.Writetool.edit_file',
cloudra/tools/registry.py:294           class_name='EditFile',
cloudra/tools/registry.py:295           tool_name='edit_file',
cloudra/tools/registry.py:296           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:297           dag_substantive=True,
cloudra/tools/registry.py:298       ),
cloudra/tools/registry.py:299       ToolRegistration(
cloudra/tools/registry.py:300           module='cloudra.tools.Writetool.text_writer',
cloudra/tools/registry.py:301           class_name='TextWriter',
cloudra/tools/registry.py:302           tool_name='text_writer',
cloudra/tools/registry.py:303           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:304           dag_substantive=True,
cloudra/tools/registry.py:305       ),
cloudra/tools/registry.py:306       ToolRegistration(
cloudra/tools/registry.py:307           module='cloudra.tools.Writetool.update_memory',
cloudra/tools/registry.py:308           class_name='UpdateMemory',
cloudra/tools/registry.py:309           tool_name='update_memory',
cloudra/tools/registry.py:310           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:311       ),
cloudra/tools/registry.py:312   )
cloudra/web_launcher.py:1   """Web frontend launcher used by the `cloudra` command.
cloudra/web_launcher.py:2
cloudra/web_launcher.py:3   Keeps Node/npm knowledge out of the CLI module.  The launcher works for source
cloudra/web_launcher.py:4   checkouts and editable installs; packaged wheels can point CLOUDRA_FRONTEND_DIR
cloudra/web_launcher.py:5   at an unpacked frontend directory.
```

```
cloudra/web_launcher.py:6   """
cloudra/web_launcher.py:7
cloudra/web_launcher.py:8   from __future__ import annotations
cloudra/web_launcher.py:9
cloudra/web_launcher.py:10  import os
cloudra/web_launcher.py:11  import shutil
cloudra/web_launcher.py:12  import socket
cloudra/web_launcher.py:13  import subprocess
cloudra/web_launcher.py:14  import sys
cloudra/web_launcher.py:15  import time
cloudra/web_launcher.py:16  import urllib.request
cloudra/web_launcher.py:17  from pathlib import Path
cloudra/web_launcher.py:18
cloudra/web_launcher.py:19
cloudra/web_launcher.py:20  def _frontend_dir() -> Path:
cloudra/web_launcher.py:21      override = os.environ.get("CLOUDRA_FRONTEND_DIR", "").strip()
cloudra/web_launcher.py:22      candidates = []
cloudra/web_launcher.py:23      if override:
cloudra/web_launcher.py:24          candidates.append(Path(override))
cloudra/web_launcher.py:25      # cloudra/web_launcher.py -> cloudra/.. == repository root (source checkout)
cloudra/web_launcher.py:26      candidates.append(Path(__file__).resolve().parents[1] / "frontend")
cloudra/web_launcher.py:27      candidates.append(Path.cwd() / "frontend")
cloudra/web_launcher.py:28      for candidate in candidates:
cloudra/web_launcher.py:29          if (candidate / "package.json").is_file():
cloudra/web_launcher.py:30              return candidate.resolve()
cloudra/web_launcher.py:31      raise RuntimeError(
cloudra/web_launcher.py:32          "frontend directory not found; set CLOUDRA_FRONTEND_DIR to the "
cloudra/web_launcher.py:33          "directory containing frontend/package.json"
cloudra/web_launcher.py:34      )
cloudra/web_launcher.py:35
cloudra/web_launcher.py:36
cloudra/web_launcher.py:37  def _check_node(frontend: Path) -> None:
cloudra/web_launcher.py:38      npm = shutil.which("npm")
cloudra/web_launcher.py:39      node = shutil.which("node")
cloudra/web_launcher.py:40      if npm is None or node is None:
cloudra/web_launcher.py:41          raise RuntimeError(
cloudra/web_launcher.py:42              "Node.js >= 18 and npm are required for the Cloudra web frontend. "
cloudra/web_launcher.py:43              f"Frontend found at {frontend}. Install from https://nodejs.org/"
cloudra/web_launcher.py:44          )
cloudra/web_launcher.py:45
cloudra/web_launcher.py:46
cloudra/web_launcher.py:47  def _ensure_frontend(frontend: Path, *, rebuild: bool = False, skip_build: bool = False) ->
None:
cloudra/web_launcher.py:48      _check_node(frontend)
cloudra/web_launcher.py:49      if not (frontend / "node_modules").is_dir():
cloudra/web_launcher.py:50          print("[cloudra] installing frontend dependencies...", flush=True)
cloudra/web_launcher.py:51          subprocess.run(["npm", "install", "--no-audit", "--no-fund"], cwd=frontend, check=True)
cloudra/web_launcher.py:52      dist = frontend / "dist"
cloudra/web_launcher.py:53      if skip_build:
cloudra/web_launcher.py:54          if not (dist / "index.html").is_file():
cloudra/web_launcher.py:55              raise RuntimeError("--skip-build was requested but frontend/dist is missing")
cloudra/web_launcher.py:56          return
cloudra/web_launcher.py:57      if rebuild or not (dist / "index.html").is_file():
```

```
cloudra/web_launcher.py:58          print("[cloudra] building frontend...", flush=True)
cloudra/web_launcher.py:59          subprocess.run(["npm", "run", "build"], cwd=frontend, check=True)
cloudra/web_launcher.py:60
cloudra/web_launcher.py:61
cloudra/web_launcher.py:62  def _port_is_bindable(preferred: int) -> bool:
cloudra/web_launcher.py:63      with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
cloudra/web_launcher.py:64          try:
cloudra/web_launcher.py:65              sock.bind(("127.0.0.1", preferred))
cloudra/web_launcher.py:66          except OSError:
cloudra/web_launcher.py:67              return False
cloudra/web_launcher.py:68          return True
cloudra/web_launcher.py:69
cloudra/web_launcher.py:70
cloudra/web_launcher.py:71  def _runtime_is_healthy(port: int) -> bool:
cloudra/web_launcher.py:72      try:
cloudra/web_launcher.py:73          with urllib.request.urlopen(
cloudra/web_launcher.py:74              f"http://127.0.0.1:{port}/api/v2/runtime/health", timeout=1.0
cloudra/web_launcher.py:75          ) as resp:
cloudra/web_launcher.py:76              return 200 <= resp.status < 300
cloudra/web_launcher.py:77      except Exception:
cloudra/web_launcher.py:78          return False
cloudra/web_launcher.py:79
cloudra/web_launcher.py:80
cloudra/web_launcher.py:81  def _runtime_serves_spa(port: int) -> bool:
cloudra/web_launcher.py:82      try:
cloudra/web_launcher.py:83          with urllib.request.urlopen(f"http://127.0.0.1:{port}/", timeout=1.0) as resp:
cloudra/web_launcher.py:84              body = resp.read(4096).decode("utf-8", "ignore")
cloudra/web_launcher.py:85              return 200 <= resp.status < 300 and '<div id="root"></div>' in body
cloudra/web_launcher.py:86      except Exception:
cloudra/web_launcher.py:87          return False
cloudra/web_launcher.py:88
cloudra/web_launcher.py:89
cloudra/web_launcher.py:90  def _find_api_port(preferred: int, env_name: str) -> tuple[int, bool]:
cloudra/web_launcher.py:91      """Return (port, reused_existing_runtime).
cloudra/web_launcher.py:92
cloudra/web_launcher.py:93      A runtime is only reused when it both serves the API and the built SPA on
cloudra/web_launcher.py:94      the same port; otherwise we start a fresh single-port runtime on a free
cloudra/web_launcher.py:95      port so the frontend never points at a stale or foreign server.
cloudra/web_launcher.py:96      """
cloudra/web_launcher.py:97      env_value = os.environ.get(env_name)
cloudra/web_launcher.py:98      if env_value:
cloudra/web_launcher.py:99          port = int(env_value)
cloudra/web_launcher.py:100         return port, _runtime_is_healthy(port) and _runtime_serves_spa(port)
cloudra/web_launcher.py:101     if _port_is_bindable(preferred):
cloudra/web_launcher.py:102         return preferred, False
cloudra/web_launcher.py:103     if _runtime_is_healthy(preferred) and _runtime_serves_spa(preferred):
cloudra/web_launcher.py:104         return preferred, True
cloudra/web_launcher.py:105     for port in range(preferred + 1, preferred + 100):
cloudra/web_launcher.py:106         if _port_is_bindable(port):
cloudra/web_launcher.py:107             return port, False
cloudra/web_launcher.py:108     raise RuntimeError(f"no free port near {preferred}; set {env_name} explicitly")
cloudra/web_launcher.py:109
```

```
cloudra/web_launcher.py:110
cloudra/web_launcher.py:111 def _ensure_electron(frontend: Path) -> None:
cloudra/web_launcher.py:112     """npm install can finish with a broken electron package when its
cloudra/web_launcher.py:113     postinstall download was interrupted.  Re-run electron's install script
cloudra/web_launcher.py:114     and verify the binary before launching."""
cloudra/web_launcher.py:115     electron_dir = frontend / "node_modules" / "electron"
cloudra/web_launcher.py:116     binary = electron_dir / "dist" / "electron"
cloudra/web_launcher.py:117     if binary.is_file() and os.access(binary, os.X_OK):
cloudra/web_launcher.py:118         return
cloudra/web_launcher.py:119     print("[cloudra] electron binary missing, repairing electron install...", flush=True)
cloudra/web_launcher.py:120     if not (electron_dir / "install.js").is_file():
cloudra/web_launcher.py:121         subprocess.run(["npm", "install", "--no-audit", "--no-fund"], cwd=frontend, check=True)
cloudra/web_launcher.py:122     subprocess.run(["node", "install.js"], cwd=electron_dir, check=True)
cloudra/web_launcher.py:123     if not (binary.is_file() and os.access(binary, os.X_OK)):
cloudra/web_launcher.py:124         raise RuntimeError(
cloudra/web_launcher.py:125             "electron failed to install its binary; run: "
cloudra/web_launcher.py:126             f"rm -rf {electron_dir} && npm install --prefix {frontend}"
cloudra/web_launcher.py:127         )
cloudra/web_launcher.py:128
cloudra/web_launcher.py:129
cloudra/web_launcher.py:130 def _wait_healthy(url: str, timeout: float = 30.0) -> None:
cloudra/web_launcher.py:131     deadline = time.monotonic() + timeout
cloudra/web_launcher.py:132     while time.monotonic() < deadline:
cloudra/web_launcher.py:133         try:
cloudra/web_launcher.py:134             with urllib.request.urlopen(url, timeout=1.0) as resp:
cloudra/web_launcher.py:135                 if 200 <= resp.status < 300:
cloudra/web_launcher.py:136                     return
cloudra/web_launcher.py:137         except Exception:
cloudra/web_launcher.py:138             pass
cloudra/web_launcher.py:139         time.sleep(0.25)
cloudra/web_launcher.py:140     raise RuntimeError(f"runtime did not become healthy at {url}")
cloudra/web_launcher.py:141
cloudra/web_launcher.py:142
cloudra/web_launcher.py:143 def launch_electron(frontend: Path, *, rebuild: bool, skip_build: bool) -> int:
cloudra/web_launcher.py:144     _ensure_frontend(frontend, rebuild=rebuild, skip_build=skip_build)
cloudra/web_launcher.py:145     _ensure_electron(frontend)
cloudra/web_launcher.py:146     env = os.environ.copy()
cloudra/web_launcher.py:147     # Electron main spawns `python3`; point it at the interpreter that is
cloudra/web_launcher.py:148     # running this CLI so venvs work out of the box.
cloudra/web_launcher.py:149     env["CLOUDRA_PYTHON"] = sys.executable
cloudra/web_launcher.py:150     print("[cloudra] starting Electron desktop frontend...", flush=True)
cloudra/web_launcher.py:151     return subprocess.call(["npm", "run", "electron"], cwd=frontend, env=env)
cloudra/web_launcher.py:152
cloudra/web_launcher.py:153
cloudra/web_launcher.py:154 def launch_browser(frontend: Path, *, rebuild: bool, skip_build: bool) -> int:
cloudra/web_launcher.py:155     api_port, reused_runtime = _find_api_port(8000, "CLOUDRA_API_PORT")
cloudra/web_launcher.py:156     _ensure_frontend(frontend, rebuild=rebuild, skip_build=skip_build)
cloudra/web_launcher.py:157     from cloudra.infra.config import get_config
cloudra/web_launcher.py:158
cloudra/web_launcher.py:159     store_path = os.environ.get(
cloudra/web_launcher.py:160         "CLOUDRA_RUNTIME_STORE",
cloudra/web_launcher.py:161         str(Path(get_config().data_dir) / "runtime-v2.db"),
```

```
cloudra/web_launcher.py:162     )
cloudra/web_launcher.py:163     env = os.environ.copy()
cloudra/web_launcher.py:164     env["CLOUDRA_RUNTIME_STORE"] = store_path
cloudra/web_launcher.py:165     env["CLOUDRA_FRONTEND_SERVE_DIR"] = str(frontend / "dist")
cloudra/web_launcher.py:166     runtime = None
cloudra/web_launcher.py:167     if reused_runtime:
cloudra/web_launcher.py:168         print(f"[cloudra] reusing existing Cloudra runtime on port {api_port}", flush=True)
cloudra/web_launcher.py:169     else:
cloudra/web_launcher.py:170         runtime = subprocess.Popen(
cloudra/web_launcher.py:171             [
cloudra/web_launcher.py:172                 sys.executable,
cloudra/web_launcher.py:173                 "-m",
cloudra/web_launcher.py:174                 "uvicorn",
cloudra/web_launcher.py:175                 "cloudra.services.api_v2:create_default_v2_app",
cloudra/web_launcher.py:176                 "--factory",
cloudra/web_launcher.py:177                 "--host",
cloudra/web_launcher.py:178                 "127.0.0.1",
cloudra/web_launcher.py:179                 "--port",
cloudra/web_launcher.py:180                 str(api_port),
cloudra/web_launcher.py:181                 "--log-level",
cloudra/web_launcher.py:182                 "warning",
cloudra/web_launcher.py:183             ],
cloudra/web_launcher.py:184             env=env,
cloudra/web_launcher.py:185         )
cloudra/web_launcher.py:186     try:
cloudra/web_launcher.py:187         _wait_healthy(f"http://127.0.0.1:{api_port}/api/v2/runtime/health")
cloudra/web_launcher.py:188         url = f"http://127.0.0.1:{api_port}"
cloudra/web_launcher.py:189         print(f"[cloudra] web frontend: {url}", flush=True)
cloudra/web_launcher.py:190         print(f"[cloudra] open {url} in your browser", flush=True)
cloudra/web_launcher.py:191         return runtime.wait() if runtime is not None else 0
cloudra/web_launcher.py:192     except KeyboardInterrupt:
cloudra/web_launcher.py:193         return 0
cloudra/web_launcher.py:194     finally:
cloudra/web_launcher.py:195         if runtime is not None and runtime.poll() is None:
cloudra/web_launcher.py:196             runtime.terminate()
```